TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ ĐIỀU HÀNH

TIỂU LUẬN MÔN HỆ ĐIỀU HÀNH

ĐỀ TÀI:

Phân tích cơ chế quản lý và truyền thông liên tiến trình trong Hệ điều hành.

Giảng viên hướng dẫn: Ths. Nguyễn Kiên Cường

Lớp học phần: 010100174214-15DHTH09

Sinh viên thực hiện:

1. Trần Gia Bảo – 2001240040 (Nhóm trưởng)
2. Lê Quang Bảo – 2001240030

Tp. HCM, tháng 3 năm 2026

KẾ HOẠCH LÀM VIỆC CỦA NHÓM

TT	Sinh viên	MSSV	Phân công
1	Lê Quang Bảo	2001240030	Phụ trách các nội dung về thao tác tiến trình, kiến trúc đa tiến trình, cơ chế IPC và phần minh họa ví dụ.
2	Trần Gia Bảo	2001240040	Phụ trách các nội dung về khái niệm tiến trình, trạng thái tiến trình, định thời CPU và chuyển ngữ cảnh.

MỨC ĐỘ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN

TT	Sinh viên	MSSV	Mức độ đóng góp
1.	Lê Quang Bảo	2001240030	50%
2.	Trần Gia Bảo (Nhóm trưởng)	2001240040	50%
Tổng			**100%**

LỜI CẢM ƠN

Để hoàn thành tiểu luận nghiên cứu: "Phân tích cơ chế quản lý và truyền thông liên tiến trình trong Hệ điều hành" này, chúng em đã nhận được sự hỗ trợ, hướng dẫn vô cùng quý báu từ nhiều cá nhân và tập thể.

Đầu tiên và quan trọng nhất, chúng em xin bày tỏ lòng biết ơn sâu sắc và lời cảm ơn chân thành nhất đến thầy ThS. Nguyễn Kiên Cường. Thầy đã tận tình truyền đạt kiến thức, dành thời gian quý báu để hướng dẫn, gợi mở vấn đề, và chỉnh sửa nội dung trong suốt quá trình chúng em thực hiện đề tài. Những ý kiến đóng góp, nhận xét và sự hướng dẫn tận tình của Thầy là yếu tố then chốt giúp chúng em hoàn thành tiểu luận này một cách tốt nhất.

Em nhóm trưởng xin chân thành cảm ơn tất cả các thành viên trong nhóm đã cùng nhau hợp tác, nỗ lực hết mình, đoàn kết, và dành nhiều thời gian, tâm huyết để hoàn thành tiểu luận này.

Mặc dù đã cố gắng hết sức, tiểu luận chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy và các bạn để đề tài của chúng em được hoàn thiện hơn.

Nhóm

Xin chân thành cảm ơn!

MỤC LỤC

KẾ HOẠCH LÀM VIỆC CỦA NHÓM i
MỨC ĐỘ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN i
LỜI CẢM ƠN ii
MỤC LỤC 1
MỤC LỤC HÌNH 3
MỞ ĐẦU 1
PHẦN 1: KHÁI QUÁT VỀ TIẾN TRÌNH VÀ CẤU TRÚC 2
1.1 Khái niệm tiến trình 2
1.2 Các thành phần cấu thành một tiến trình 2
Nhiệm vụ thảo luận 4
1.3 Bản chất của Address Space và việc chia sẻ bộ nhớ 4
PHẦN 2: VÒNG ĐỜI VÀ QUẢN LÝ TRẠNG THÁI TIẾN TRÌNH 6
2.1 Các trạng thái của tiến trình 6
2.2 Sơ đồ chuyển trạng thái tiến trình 7
Nhiệm vụ thảo luận 7
2.3 Ví dụ thực tế về chuyển trạng thái 8
2.4 Khối điều khiển tiến trình (PCB) 9
2.5 Điều phối tiến trình 10
PHẦN 3: ĐỊNH THỜI TIẾN TRÌNH VÀ CHUYỂN NGỮ CẢNH 11
3.1 Các hàng đợi trong hệ điều hành 11
3.2 Các bộ định thời trong hệ điều hành 12
Nhiệm vụ thảo luận 12
3.3 Chuyển ngữ cảnh (Context Switch) 13
Nhiệm vụ thảo luận 15
3.4 Một số thuật toán định thời tiến trình 16
PHẦN 4: THAO TÁC TIẾN TRÌNH & KIẾN TRÚC ĐA TIẾN TRÌNH 18
4.1 Cơ chế tạo tiến trình cha – con 18
Nhiệm vụ thảo luận 19

Nhiệm vụ thảo luận 20
1. Giả lập quá trình thực thi chương trình 20
2. Giá trị trả về của biến pid 20
3. Kết quả hiển thị trên Terminal (trường hợp bình thường) 22
4. Điều gì xảy ra nếu bỏ wait(NULL)? 22
5. Kết luận 24
4.2 Tiến trình Zombie và Orphan 24
Nhiệm vụ thảo luận 25
1. Rủi ro tài nguyên đối với hệ thống nếu có quá nhiều Zombie process xuất hiện cùng lúc 25
2. Hệ điều hành chủ động xử lý và "nhận nuôi" tiến trình Orphan như thế nào? 26
3. Liên hệ giữa Zombie và Orphan 28
4. Kết luận 28
4.3 Nghiên cứu điển hình: Google Chrome 28
Nhiệm vụ thảo luận 30
PHẦN 5: TRUYỀN THÔNG LIÊN TIẾN TRÌNH (IPC) 35
5.1 Lý do cần IPC 35
5.2 Mô hình bộ nhớ chia sẻ (Shared Memory) 35
5.3 Mô hình truyền thông thông điệp (Message Passing) 36
Nhiệm vụ thảo luận 38
1. Vì sao Shared Memory có băng thông cao hơn? 38
2. Phân tích theo góc độ chi phí hệ thống 39
3. Điểm mạnh và điểm yếu của từng mô hình 39
4. Kết luận 40
5.4 So sánh hai mô hình IPC 41
Nhiệm vụ thảo luận 41
1. Hệ thống Chat nội bộ (gửi text ngắn gọn liên tục) 41
2. Trình biên tập Video (nhiều tiến trình cùng thao tác trên một mảng pixel chung) 42
3. Nhận xét mở rộng 44
4. Kết luận cuối cùng 44

5.5 PHẦN MINH HỌA/ THỰC HÀNH 45
TỔNG HỢP CÁC NHIỆM VỤ THẢO LUẬN 46
KẾT LUẬN 49
TÀI LIỆU THAM KHẢO 50

MỤC LỤC HÌNH

Hình 1. Cấu trúc bộ nhớ của một tiến trình trong hệ điều hành. 3
Hình 2. Sơ đồ chuyển trạng thái tiến trình 7
Hình 3. Cấu trúc khối điều khiển tiến trình (PCB) 9
Hình 4. Code minh họa Phân biệt CPU-bound và I/O-bound 10
Hình 5. Mối quan hệ giữa các hàng đợi và CPU trong hệ điều hành 11
Hình 6. Minh họa quá trình chuyển ngữ cảnh giữa hai tiến trình 14
Hình 7. Code minh họa First Come, First Served (FCFS) 16
Hình 8. Code minh họa Round Robin 17
Hình 9. Code minh họa Priority Scheduling 17
Hình 10. Sơ đồ chu trình tạo tiến trình bằng lệnh fork() và exec() 18
Hình 11. Code minh họa quá trình thực thi chương trình 20
Hình 12. Tiến trình Zombie 24
Hình 13. Tiến trình Orphan 25
Hình 14. Mô hình bộ nhớ chia sẻ (Shared Memory) 35
Hình 15. Mô hình truyền thông điệp (Message Passing) 37

MỞ ĐẦU

Trong bối cảnh các hệ thống máy tính ngày càng phát triển theo hướng đa nhiệm và đa người dùng, hệ điều hành đóng vai trò trung tâm trong việc quản lý và điều phối tài nguyên hệ thống. Một trong những nhiệm vụ quan trọng nhất của hệ điều hành là quản lý tiến trình các chương trình đang được thực thi – sao cho nhiều tiến trình có thể cùng tồn tại, hoạt động hiệu quả và không gây xung đột lẫn nhau.

Bên cạnh việc quản lý vòng đời và trạng thái của tiến trình, hệ điều hành còn phải cung cấp các cơ chế truyền thông liên tiến trình (Inter-Process Communication – IPC) nhằm cho phép các tiến trình trao đổi dữ liệu, phối hợp xử lý công việc và tận dụng tối đa năng lực tính toán của hệ thống. Việc lựa chọn và triển khai cơ chế IPC phù hợp có ảnh hưởng trực tiếp đến hiệu năng, tính an toàn và độ ổn định của toàn bộ hệ thống.

Xuất phát từ những yêu cầu đó, bài tiểu luận này tập trung phân tích cơ chế quản lý tiến trình trong hệ điều hành, bao gồm khái niệm tiến trình, cấu trúc, vòng đời, định thời CPU và các thao tác liên quan đến tiến trình. Đồng thời, bài viết cũng đi sâu nghiên cứu các mô hình truyền thông liên tiến trình phổ biến như bộ nhớ chia sẻ và truyền thông thông điệp, qua đó làm rõ vai trò của IPC trong các hệ thống hiện đại. Thông qua việc kết hợp giữa lý thuyết và các ví dụ minh họa, bài tham luận nhằm giúp người học có cái nhìn toàn diện hơn về cách hệ điều hành vận hành trong môi trường đa tiến trình.

PHẦN 1: KHÁI QUÁT VỀ TIẾN TRÌNH VÀ CẤU TRÚC

1.1 Khái niệm tiến trình

Trong hệ điều hành, **tiến trình (process)** được hiểu là một **chương trình đang được thực thi** cùng với toàn bộ trạng thái và tài nguyên mà nó sử dụng trong quá trình hoạt động. Tiến trình không chỉ bao gồm mã lệnh của chương trình mà còn chứa các thông tin cần thiết để hệ điều hành có thể quản lý và điều phối việc thực thi chương trình đó trên CPU.

Cần phân biệt rõ giữa **chương trình** và **tiến trình**:

> **Chương trình (program)** là một tập hợp các câu lệnh được lưu trữ trên thiết bị lưu trữ như ổ cứng hoặc SSD. Chương trình mang tính **thụ động**, chỉ tồn tại dưới dạng tệp tin và không tiêu thụ tài nguyên CPU khi chưa được thực thi.
>
> **Tiến trình (process)** là một thể hiện cụ thể của chương trình khi nó đang chạy. Tiến trình mang tính **chủ động**, có trạng thái, chiếm dụng tài nguyên hệ thống như CPU, bộ nhớ, thiết bị vào/ra và có thể tương tác với các tiến trình khác.

Một chương trình có thể được thực thi nhiều lần, và mỗi lần thực thi sẽ tạo ra một tiến trình riêng biệt. Các tiến trình này có thể cùng sử dụng chung mã chương trình nhưng lại có không gian bộ nhớ và trạng thái thực thi độc lập với nhau.

1.2 Các thành phần cấu thành một tiến trình

Một tiến trình trong hệ điều hành bao gồm nhiều thành phần khác nhau, phản ánh đầy đủ ngữ cảnh thực thi của chương trình. Các thành phần chính bao gồm:

a. Text (Code Segment)

Text là vùng bộ nhớ chứa mã lệnh của chương trình. Đây là phần không thay đổi trong suốt quá trình thực thi và thường được chia sẻ giữa các tiến trình được tạo ra từ cùng một chương trình nhằm tiết kiệm bộ nhớ.

b. Program Counter (Bộ đếm chương trình)

Program Counter lưu địa chỉ của lệnh tiếp theo sẽ được CPU thực thi. Khi tiến trình bị gián đoạn hoặc xảy ra chuyển ngữ cảnh, giá trị của Program Counter được lưu lại để tiến trình có thể tiếp tục thực thi đúng vị trí khi được cấp CPU trở lại.

c. Registers (Thanh ghi)

Thanh ghi là các vùng lưu trữ tốc độ cao bên trong CPU, dùng để lưu trữ dữ liệu tạm thời, kết quả tính toán trung gian và các thông tin điều khiển. Trạng thái của các thanh ghi là một phần quan trọng trong ngữ cảnh của tiến trình.

d. Stack (Ngăn xếp)

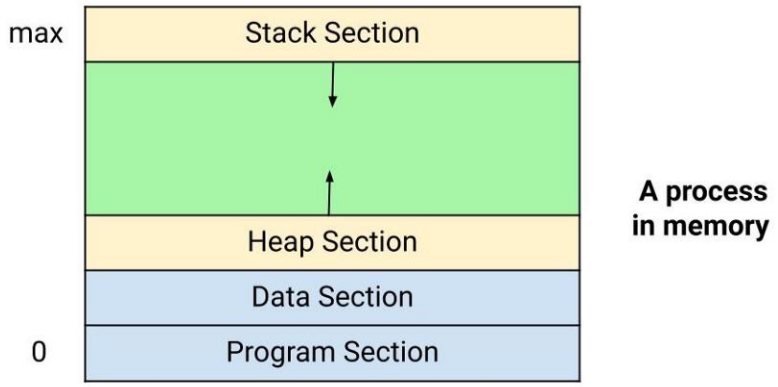
Stack được sử dụng để lưu trữ thông tin liên quan đến lời gọi hàm, bao gồm tham số truyền vào, biến cục bộ và địa chỉ trả về sau khi hàm kết thúc. Stack hoạt động theo nguyên tắc vào sau ra trước (LIFO).

e. Data Section (Vùng dữ liệu)

Data section chứa các biến toàn cục và biến tĩnh của chương trình. Các biến này tồn tại trong suốt vòng đời của tiến trình và có thể được truy cập bởi nhiều hàm trong chương trình.

f. Heap (Khối xếp)

Heap là vùng bộ nhớ dùng cho cấp phát động trong quá trình thực thi, ví dụ như khi sử dụng các hàm malloc() trong C. Dung lượng của heap có thể thay đổi trong suốt vòng đời của tiến trình tùy theo nhu cầu sử dụng bộ nhớ.



Hình 1. Cấu trúc bộ nhớ của một tiến trình trong hệ điều hành.

Nhiệm vụ thảo luận

Dựa vào sơ đồ bộ nhớ của tiến trình, giải thích tại sao hai người dùng cùng chạy chung một file .exe lại tạo ra hai tiến trình với vùng Stack/Heap hoàn toàn khác nhau? Điều gì xảy ra trong vùng Data?

Một chương trình .exe chỉ là tệp chứa mã lệnh được lưu trên đĩa. Khi người dùng chạy chương trình, hệ điều hành sẽ nạp mã lệnh vào bộ nhớ và tạo ra một tiến trình (process) riêng cho mỗi lần thực thi. Mỗi tiến trình sẽ được cấp phát một không gian bộ nhớ riêng, bao gồm các vùng như Code, Data, Heap và Stack.

Vùng Stack được sử dụng để lưu trữ các biến cục bộ, tham số hàm và địa chỉ trả về của chương trình trong quá trình thực thi. Vùng Heap được dùng để cấp phát bộ nhớ động trong khi chương trình chạy. Vì mỗi tiến trình có không gian bộ nhớ riêng nên Stack và Heap của các tiến trình sẽ không trùng nhau, ngay cả khi chúng chạy cùng một chương trình.

Đối với vùng Data, đây là nơi lưu trữ các biến toàn cục và biến tĩnh của chương trình. Khi nhiều tiến trình chạy cùng một chương trình, mỗi tiến trình vẫn có bản sao riêng của vùng Data trong không gian bộ nhớ của mình. Nhờ vậy, dữ liệu của tiến trình này sẽ không ảnh hưởng đến dữ liệu của tiến trình khác, giúp hệ điều hành đảm bảo tính độc lập và an toàn giữa các tiến trình.

1.3 Bản chất của Address Space và việc chia sẻ bộ nhớ

Một đặc điểm quan trọng của tiến trình là mỗi tiến trình có một không gian địa chỉ (Address Space) riêng biệt. Điều này có nghĩa là các tiến trình không thể truy cập trực tiếp vào vùng nhớ của nhau.

Ví dụ, nếu hai người dùng cùng chạy một chương trình program.exe, hệ điều hành sẽ tạo ra hai tiến trình khác nhau. Mỗi tiến trình sẽ có:

- Stack riêng
- Heap riêng
- Data riêng

Do đó các biến cục bộ, biến toàn cục hoặc bộ nhớ cấp phát động của mỗi tiến trình đều độc lập.

Tuy nhiên, vùng Text (code) có thể được chia sẻ giữa nhiều tiến trình. Nguyên nhân là vì mã lệnh của chương trình thường không thay đổi trong quá trình thực thi và được đặt ở chế độ chỉ đọc. Nhờ vậy hệ điều hành có thể cho nhiều tiến trình sử dụng chung vùng mã này để tiết kiệm bộ nhớ vật lý.

Cơ chế này được thực hiện thông qua bộ nhớ ảo và bảng trang (page table) của từng tiến trình.

PHẦN 2: VÒNG ĐỜI VÀ QUẢN LÝ TRẠNG THÁI TIẾN TRÌNH

2.1 Các trạng thái của tiến trình

Trong quá trình tồn tại và thực thi, một tiến trình không hoạt động liên tục ở một trạng thái duy nhất mà sẽ chuyển đổi qua nhiều trạng thái khác nhau. Hệ điều hành sử dụng các trạng thái này để theo dõi và quản lý việc thực thi tiến trình một cách hiệu quả. Thông thường, một tiến trình trải qua năm trạng thái cơ bản sau:

a. New (Mới tạo)

Trạng thái *New* là trạng thái khởi đầu của tiến trình, khi tiến trình vừa được tạo ra nhưng chưa được cấp phát tài nguyên đầy đủ và chưa được đưa vào hàng đợi sẵn sàng. Ở giai đoạn này, hệ điều hành thực hiện các thao tác khởi tạo cần thiết như tạo PCB và chuẩn bị không gian bộ nhớ cho tiến trình.

b. Ready (Sẵn sàng)

Tiến trình ở trạng thái *Ready* đã sẵn sàng để thực thi và đang chờ được cấp CPU. Tại thời điểm này, tiến trình đã có đầy đủ tài nguyên cần thiết, ngoại trừ CPU. Các tiến trình ở trạng thái Ready được lưu trong **Ready Queue**.

c. Running (Đang chạy)

Khi được bộ định thời cấp CPU, tiến trình chuyển sang trạng thái *Running*. Trong trạng thái này, CPU đang thực thi các lệnh của tiến trình. Mỗi thời điểm, trong hệ thống chỉ có một tiến trình ở trạng thái Running trên một CPU.

d. Waiting (Blocked)

Tiến trình chuyển sang trạng thái *Waiting* khi nó cần chờ một sự kiện nào đó xảy ra, chẳng hạn như hoàn thành thao tác vào/ra (I/O), nhận dữ liệu từ thiết bị hoặc chờ tín hiệu từ tiến trình khác. Trong trạng thái này, tiến trình tạm thời không thể tiếp tục thực thi dù CPU có sẵn.

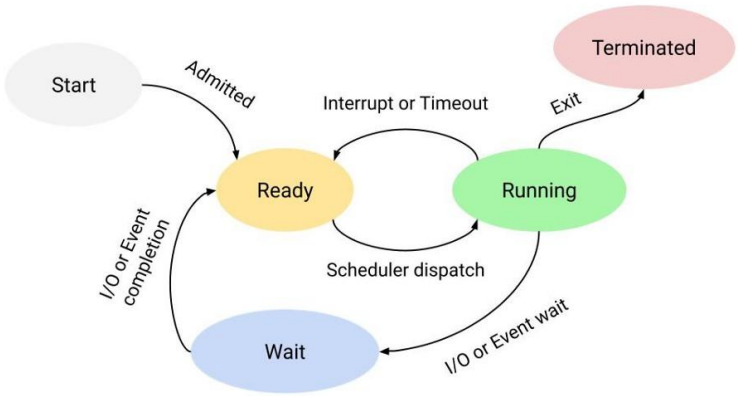
e. Terminated (Kết thúc)

Trạng thái *Terminated* là trạng thái kết thúc của tiến trình. Khi tiến trình hoàn thành nhiệm vụ hoặc bị hệ điều hành kết thúc, các tài nguyên mà tiến trình sử dụng sẽ được thu hồi, PCB của tiến trình sẽ bị xóa khỏi hệ thống.

2.2 Sơ đồ chuyển trạng thái tiến trình

Sự chuyển đổi giữa các trạng thái của tiến trình được điều khiển bởi hệ điều hành thông qua bộ định thời và các sự kiện hệ thống. Quá trình chuyển trạng thái phổ biến bao gồm:

- *New* → *Ready*: Tiến trình được khởi tạo và đưa vào hệ thống.
- *Ready* → *Running*: Tiến trình được bộ định thời cấp CPU.
- *Running* → *Waiting*: Tiến trình yêu cầu thao tác I/O hoặc chờ sự kiện.
- *Waiting* → *Ready*: Sự kiện cần thiết đã hoàn thành.



Hình 2. Sơ đồ chuyển trạng thái tiến trình

- *Running* → *Ready*: Tiến trình bị ngắt do hết thời gian CPU hoặc bị tiến trình ưu tiên cao hơn chiếm CPU.
- *Running* → *Terminated*: Tiến trình kết thúc thực thi.

Nhiệm vụ thảo luận

Xác định 3 sự kiện thực tế trong quá trình sử dụng máy tính (ví dụ: người dùng gõ phím, tải file hoàn tất, mở tab mới) tương ứng với các mũi tên chuyển trạng thái từ "Running" sang "Waiting" và từ "Waiting" sang "Ready" trong vòng đời tiến trình.

Trong quá trình hoạt động của hệ điều hành, một tiến trình có thể chuyển đổi giữa nhiều trạng thái khác nhau tùy thuộc vào các sự kiện xảy ra trong hệ thống. Một số ví dụ thực tế có thể minh họa cho các sự chuyển trạng thái này như sau:

Ví dụ 1: Tải file từ Internet

- Khi chương trình đang chạy để tải file → tiến trình ở trạng thái Running.
- Khi chương trình phải chờ dữ liệu từ mạng → tiến trình chuyển sang trạng thái Waiting.
- Khi dữ liệu được tải xong và hệ điều hành thông báo hoàn tất → tiến trình chuyển sang Ready để tiếp tục xử lý dữ liệu.

Ví dụ 2: Người dùng mở một tab mới trong trình duyệt

- Trình duyệt đang xử lý thao tác của người dùng → Running.
- Khi trình duyệt cần chờ dữ liệu từ máy chủ web → Waiting.
- Khi dữ liệu trang web được nhận về → tiến trình quay lại Ready để tiếp tục hiển thị nội dung.

Ví dụ 3: Người dùng gõ phím trong chương trình

- Chương trình đang chạy và xử lý dữ liệu → Running.
- Khi chương trình chờ người dùng nhập dữ liệu từ bàn phím → Waiting.

Khi người dùng nhập xong và hệ điều hành gửi tín hiệu hoàn tất sự kiện → tiến trình chuyển sang Ready để tiếp tục thực thi.

2.3 Ví dụ thực tế về chuyển trạng thái

Ví dụ khi người dùng chạy một chương trình nhập dữ liệu từ bàn phím:

1. Chương trình được tạo → **New**
2. Được đưa vào hàng đợi CPU → **Ready**
3. CPU cấp quyền thực thi → **Running**
4. Chương trình yêu cầu nhập dữ liệu → **Waiting**
5. Sau khi người dùng nhập xong → quay lại **Ready**
6. CPU tiếp tục xử lý → **Running**
7. Chương trình kết thúc → **Terminated**

Ví dụ này cho thấy tiến trình liên tục thay đổi trạng thái trong quá trình thực thi.

2.4 Khối điều khiển tiến trình (PCB)

a. Chức năng của PCB

Khối điều khiển tiến trình (Process Control Block – PCB) là cấu trúc dữ liệu do hệ điều hành tạo ra để quản lý mỗi tiến trình. PCB đóng vai trò như "hồ sơ" của tiến trình, lưu trữ toàn bộ thông tin cần thiết để hệ điều hành có thể theo dõi, điều khiển và phục hồi tiến trình trong quá trình thực thi.

PCB đặc biệt quan trọng trong cơ chế **chuyển ngữ cảnh (context switch)**, khi CPU chuyển từ tiến trình này sang tiến trình khác. Lúc này, trạng thái của tiến trình đang chạy sẽ được lưu vào PCB để có thể tiếp tục thực thi chính xác tại thời điểm sau.

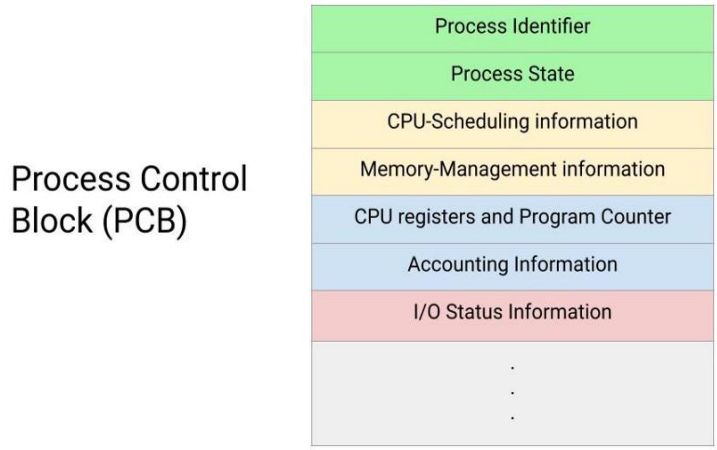
b. Các thông tin được lưu trong PCB

Một PCB thường bao gồm các thông tin sau:

Process ID (PID): Mã định danh duy nhất của tiến trình.

Process State: Trạng thái hiện tại của tiến trình (New, Ready, Running, Waiting, Terminated).

Program Counter: Địa chỉ của lệnh tiếp theo sẽ được thực thi.



Hình 3. Cấu trúc khối điều khiển tiến trình (PCB)

CPU Registers: Giá trị các thanh ghi của CPU.

CPU Scheduling Information: Thông tin liên quan đến định thời như độ ưu tiên, thời gian CPU đã sử dụng.

Memory Management Information: Thông tin về không gian bộ nhớ của tiến trình.

I/O Status Information: Danh sách các thiết bị vào/ra mà tiến trình đang sử dụng.

2.5 Điều phối tiến trình

Hệ điều hành sử dụng các bộ lập lịch (scheduler) để điều phối tiến trình.

Long-term scheduler

Quyết định tiến trình nào được đưa vào hệ thống.

Short-term scheduler

Chọn tiến trình tiếp theo để cấp CPU.

Một nhiệm vụ quan trọng của bộ lập lịch dài hạn là **cân bằng giữa các tiến trình CPU-bound và I/O-bound**:

- CPU-bound: sử dụng nhiều CPU
- I/O-bound: thường xuyên chờ thiết bị I/O

Sự cân bằng này giúp hệ thống hoạt động hiệu quả hơn.

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>

// Mô phỏng tiến trình CPU-bound: Tính số nguyên tố (tận dụng tối đa CPU)
void cpuBoundTask(int id) {
    std::cout << "Process " << id << " (CPU-bound) starts computing...\n";
    long long count = 0;
    for (long long i = 0; i < 1e8; ++i) { count += i; } // Tính toán nặng
    std::cout << "Process " << id << " (CPU-bound) finished.\n";
}

// Mô phỏng tiến trình I/O-bound: Chờ người dùng hoặc đọc file (CPU nghỉ)
void ioBoundTask(int id) {
    std::cout << "Process " << id << " (I/O-bound) waiting for I/O device...\n";
    std::this_thread::sleep_for(std::chrono::milliseconds(500)); // Giả lập chờ I/O
    std::cout << "Process " << id << " (I/O-bound) received data and finished.\n";
}
```

Hình 4. Code minh họa Phân biệt CPU-bound và I/O-bound

PHẦN 3: ĐỊNH THỜI TIẾN TRÌNH VÀ CHUYỂN NGỮ CẢNH

3.1 Các hàng đợi trong hệ điều hành

Để quản lý và điều phối việc thực thi các tiến trình một cách hiệu quả, hệ điều hành sử dụng nhiều loại hàng đợi khác nhau. Mỗi hàng đợi phản ánh trạng thái và nhu cầu tài nguyên của các tiến trình tại từng thời điểm.

a. Job Queue

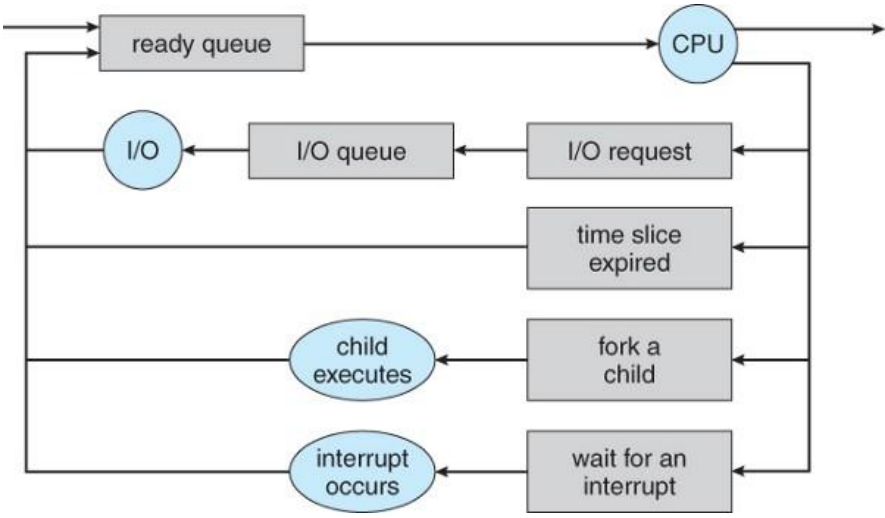
Job queue là hàng đợi chứa tất cả các tiến trình đang tồn tại trong hệ thống. Khi một tiến trình mới được tạo, nó sẽ được đưa vào job queue và chờ được bộ định thời dài hạn xem xét để đưa vào hệ thống thực thi.

b. Ready Queue

Ready queue chứa các tiến trình đã sẵn sàng để thực thi và đang chờ được cấp CPU. Các tiến trình trong hàng đợi này đã được cấp phát đầy đủ tài nguyên cần thiết, ngoại trừ CPU. Bộ định thời ngắn hạn sẽ lựa chọn một tiến trình từ ready queue để đưa vào trạng thái Running.

c. Device Queue

Device queue là hàng đợi chứa các tiến trình đang chờ hoàn thành thao tác vào/ra (I/O) từ các thiết bị như ổ đĩa, máy in hoặc thiết bị mạng. Mỗi thiết bị I/O thường có một device queue riêng để quản lý các tiến trình đang chờ thiết bị đó.



Hình 5. Mối quan hệ giữa các hàng đợi và CPU trong hệ điều hành

3.2 Các bộ định thời trong hệ điều hành

Hệ điều hành sử dụng các bộ định thời (scheduler) để quyết định tiến trình nào sẽ được thực thi, vào thời điểm nào và trong bao lâu. Mỗi loại bộ định thời đảm nhận một vai trò khác nhau.

a. Long-term Scheduler (Bộ định thời dài hạn)

Bộ định thời dài hạn quyết định tiến trình nào được đưa vào hệ thống để thực thi. Nó kiểm soát mức độ đa chương trình (degree of multiprogramming) và ảnh hưởng đến tải tổng thể của hệ thống. Bộ định thời này không được gọi thường xuyên.

b. Short-term Scheduler (Bộ định thời ngắn hạn)

Bộ định thời ngắn hạn quyết định tiến trình nào trong ready queue sẽ được cấp CPU tiếp theo. Đây là bộ định thời hoạt động thường xuyên nhất và có ảnh hưởng trực tiếp đến hiệu năng và khả năng phản hồi của hệ thống.

c. Medium-term Scheduler (Bộ định thời trung hạn)

Bộ định thời trung hạn có nhiệm vụ tạm thời đình chỉ (suspend) một số tiến trình và đưa chúng ra khỏi bộ nhớ chính nhằm giảm tải cho hệ thống. Khi cần thiết, các tiến trình này sẽ được đưa trở lại bộ nhớ để tiếp tục thực thi.

Nhiệm vụ thảo luận

Phân tích sự khác biệt giữa tiến trình thiên hướng I/O (I/O-bound) và tiến trình thiên hướng xử lý (CPU-bound). Bộ định thời dài hạn (Long-term Scheduler) cần kết hợp tỷ lệ hai loại tiến trình này như thế nào để tối ưu hóa toàn bộ hệ thống?

Trong hệ điều hành, các tiến trình có thể được phân thành hai loại chính là CPU-bound và I/O-bound, tùy thuộc vào cách chúng sử dụng tài nguyên hệ thống.

Tiến trình CPU-bound là những tiến trình dành phần lớn thời gian để thực hiện các phép tính trên CPU. Chúng ít yêu cầu các thao tác nhập/xuất và thường sử dụng CPU liên tục trong thời gian dài.

Tiến trình I/O-bound là những tiến trình thường xuyên thực hiện các thao tác nhập/xuất như đọc ghi dữ liệu từ đĩa, mạng hoặc thiết bị ngoại vi. Do phải chờ các thiết bị này phản hồi nên các tiến trình này thường ở trạng thái Waiting nhiều hơn.

Để tối ưu hiệu suất hệ thống, Long-term Scheduler cần lựa chọn một tỷ lệ hợp lý giữa hai loại tiến trình này. Nếu hệ thống chỉ có nhiều tiến trình CPU-bound thì CPU có thể bị sử dụng liên tục nhưng các thiết bị I/O sẽ ít được sử dụng. Ngược lại, nếu quá nhiều tiến trình I/O-bound thì CPU có thể bị nhàn rỗi trong khi các tiến trình đang chờ thao tác I/O.

Vì vậy, việc kết hợp cân bằng giữa tiến trình CPU-bound và I/O-bound giúp hệ thống tận dụng tốt cả CPU và các thiết bị I/O, từ đó nâng cao hiệu suất tổng thể của hệ điều hành.

3.3 Chuyển ngữ cảnh (Context Switch)

a. Cơ chế chuyển ngữ cảnh

Chuyển ngữ cảnh xảy ra khi CPU chuyển từ việc thực thi một tiến trình sang một tiến trình khác. Quá trình này bao gồm các bước chính:

Lưu trạng thái của tiến trình đang chạy (program counter, thanh ghi, trạng thái CPU) vào PCB.

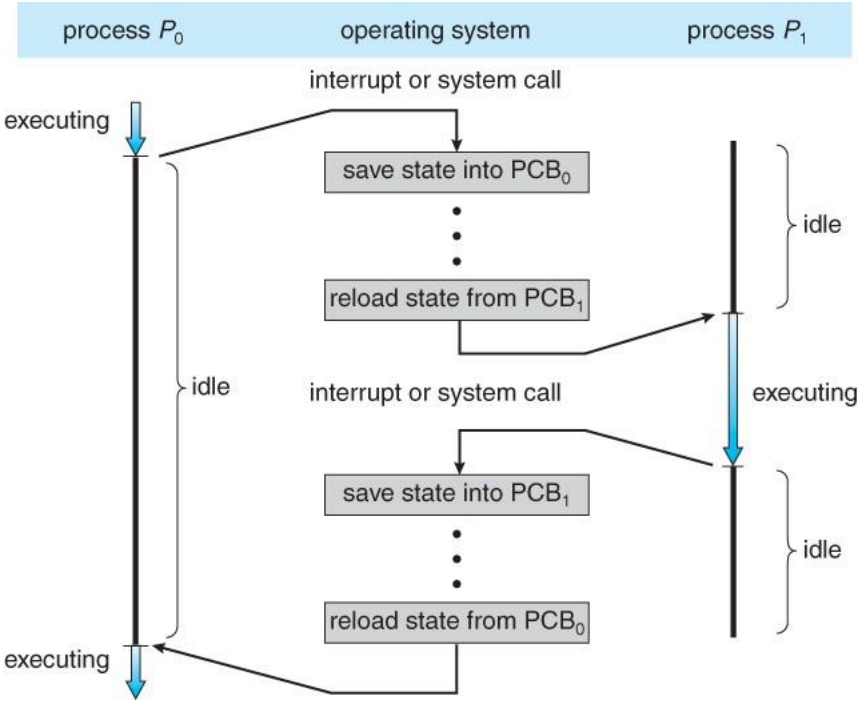
Nạp trạng thái của tiến trình mới từ PCB.

Tiếp tục thực thi tiến trình mới từ vị trí đã lưu trước đó.

Chuyển ngữ cảnh có thể xảy ra do nhiều nguyên nhân như tiến trình hết thời gian CPU, tiến trình yêu cầu I/O hoặc xuất hiện tiến trình có độ ưu tiên cao hơn.

b. Chi phí và ảnh hưởng đến hiệu năng

Chuyển ngữ cảnh không tạo ra công việc hữu ích cho người dùng nhưng lại tiêu tốn thời gian CPU. Nếu xảy ra quá thường xuyên, chi phí chuyển ngữ cảnh sẽ làm giảm hiệu năng hệ thống, tăng thời gian chờ và giảm khả năng đáp ứng. Do đó, hệ điều hành cần cân bằng giữa tính công bằng và hiệu suất khi thiết kế cơ chế định thời.



Hình 6. Minh họa quá trình chuyển ngữ cảnh giữa hai tiến trình

Hình trên minh họa quá trình chuyển ngữ cảnh (Context Switching) giữa hai tiến trình P_0 và P_1 trong hệ điều hành.

Ban đầu, tiến trình P_0 đang thực thi (executing) trên CPU. Khi xảy ra ngắt (interrupt) hoặc lời gọi hệ thống (system call), hệ điều hành sẽ tạm dừng tiến trình này. Lúc đó, hệ điều hành sẽ lưu trạng thái hiện tại của P_0 vào PCB_0 (Process Control Block), bao gồm các thông tin như thanh ghi CPU, bộ đếm chương trình (Program Counter), và các thông tin cần thiết khác.

Sau khi lưu trạng thái của P_0, hệ điều hành sẽ nạp lại trạng thái của tiến trình P_1 từ PCB_1 vào CPU, cho phép tiến trình P_1 bắt đầu thực thi. Trong thời gian này, tiến trình P_0 chuyển sang trạng thái chờ CPU (idle).

Khi có một ngắt hoặc system call khác, hệ điều hành lại thực hiện quá trình tương tự: lưu trạng thái của P_1 vào PCB_1, sau đó khôi phục trạng thái của P_0 từ PCB_0 để P_0 tiếp tục thực thi.

Quá trình lưu và khôi phục trạng thái giữa các tiến trình được gọi là chuyển ngữ cảnh (Context Switching). Đây là cơ chế quan trọng giúp hệ điều hành chia sẻ CPU giữa nhiều tiến trình, tạo cảm giác các chương trình chạy song song, mặc dù CPU chỉ thực thi một tiến trình tại một thời điểm.

Nhiệm vụ thảo luận

Trong quá trình chuyển ngữ cảnh (Context Switching), CPU có một khoảng thời gian nhàn rỗi (idle) khi hệ điều hành phải lưu và tải trạng thái của các tiến trình vào PCB. Đây được xem là một "chi phí quản lý". Hãy đề xuất 2 giải pháp (có thể từ phần cứng hoặc phần mềm) để giảm thiểu thời gian chuyển ngữ cảnh này.

Trong quá trình chuyển ngữ cảnh, hệ điều hành phải lưu trạng thái của tiến trình hiện tại vào Process Control Block (PCB) và sau đó khôi phục trạng thái của tiến trình tiếp theo. Trong khoảng thời gian này, CPU không thực hiện công việc hữu ích nên được xem là overhead của hệ thống.

Để giảm chi phí này, có thể áp dụng một số giải pháp sau:

1. Tối ưu thuật toán lập lịch CPU

Hệ điều hành có thể sử dụng các thuật toán lập lịch phù hợp như Round Robin với quantum hợp lý hoặc Priority Scheduling để giảm số lần chuyển đổi tiến trình không cần thiết. Khi số lần context switching giảm, chi phí quản lý hệ thống cũng giảm theo.

2. Cải thiện phần cứng hệ thống

Việc sử dụng CPU có tốc độ cao hơn hoặc hỗ trợ tốt hơn cho cơ chế quản lý tiến trình sẽ giúp việc lưu và khôi phục trạng thái tiến trình diễn ra nhanh hơn. Ngoài ra, bộ nhớ nhanh và cache hiệu quả cũng giúp giảm thời gian xử lý trong quá trình chuyển ngữ cảnh.

Nhờ các giải pháp này, hệ điều hành có thể giảm thời gian CPU bị nhàn rỗi và nâng cao hiệu suất tổng thể của hệ thống.

3.4 Một số thuật toán định thời tiến trình

Ngoài các khái niệm về hàng đợi và bộ định thời, hệ điều hành còn áp dụng nhiều thuật toán định thời cụ thể nhằm tối ưu việc phân phối CPU.

a. FCFS (First Come First Served)

Tiến trình nào đến trước sẽ được cấp CPU trước. Thuật toán này đơn giản nhưng có nhược điểm là có thể gây ra hiện tượng chờ đợi lâu cho các tiến trình ngắn.

```cpp
#include <iostream>
#include <vector>

struct Process {
    int id;
    int burst_time;
    int waiting_time;
    int turnaround_time;
};

void findFCFS(std::vector<Process>& processes) {
    processes[0].waiting_time = 0;

    // Tính Waiting Time
    for (int i = 1; i < processes.size(); i++) {
        processes[i].waiting_time = processes[i-1].burst_time + processes[i-1].waiting_time;
    }

    // Tính Turnaround Time (Waiting + Burst)
    std::cout << "PID\tBurst\tWaiting\tTurnaround\n";
    for (auto& p : processes) {
        p.turnaround_time = p.burst_time + p.waiting_time;
        std::cout << p.id << "\t" << p.burst_time << "\t" << p.waiting_time
                  << "\t" << p.turnaround_time << "\n";
    }
}
```

Hình 7. Code minh họa First Come, First Served (FCFS)

b. Round Robin

Mỗi tiến trình được cấp CPU trong một khoảng thời gian cố định gọi là quantum. Sau khi hết quantum, tiến trình sẽ bị đưa về ready queue nếu chưa hoàn thành. Thuật toán này phù hợp với hệ thống chia sẻ thời gian và đảm bảo tính công bằng.

```cpp
#include <iostream>
#include <vector>

void findRoundRobin(int ids[], int burst[], int n, int quantum) {
    std::vector<int> remaining_burst(burst, burst + n);
    std::vector<int> waiting_time(n, 0);
    int curr_time = 0;

    while (true) {
        bool done = true;
        for (int i = 0; i < n; i++) {
            if (remaining_burst[i] > 0) {
                done = false;
                if (remaining_burst[i] > quantum) {
                    curr_time += quantum;
                    remaining_burst[i] -= quantum;
                } else {
                    curr_time += remaining_burst[i];
                    waiting_time[i] = curr_time - burst[i];
                    remaining_burst[i] = 0;
                }
            }
        }
        if (done) break;
    }

    std::cout << "\nRound Robin (Quantum = " << quantum << "):\n";
    for (int i = 0; i < n; i++) {
        std::cout << "P" << ids[i] << " | Waiting Time: " << waiting_time[i] << "\n";
    }
```

Hình 8. Code minh họa Round Robin

c. Priority Scheduling

CPU được cấp cho tiến trình có độ ưu tiên cao nhất. Thuật toán này hiệu quả trong các hệ thống cần xử lý công việc quan trọng trước, tuy nhiên có thể gây ra hiện tượng starvation đối với các tiến trình có độ ưu tiên thấp.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

struct Process {
    int id;
    int burst_time;
    int priority; // Số càng nhỏ, độ ưu tiên càng cao
    int waiting_time;
    int turnaround_time;
};

bool comparePriority(Process a, Process b) {
    return a.priority < b.priority; // Sắp xếp theo độ ưu tiên tăng dần
}

void priorityScheduling(std::vector<Process>& proc) {
    // Sắp xếp các tiến trình theo độ ưu tiên
    std::sort(proc.begin(), proc.end(), comparePriority);

    proc[0].waiting_time = 0;
    for (int i = 1; i < proc.size(); i++) {
        proc[i].waiting_time = proc[i - 1].burst_time + proc[i - 1].waiting_time;
    }

    std::cout << "\n--- Non-preemptive Priority Scheduling ---\n";
    std::cout << "PID\tPriority\tBurst\tWaiting\tTurnaround\n";
    for (auto& p : proc) {
        p.turnaround_time = p.burst_time + p.waiting_time;
        std::cout << p.id << "\t" << p.priority << "\t\t" << p.burst_time
                  << "\t" << p.waiting_time << "\t" << p.turnaround_time << "\n";
    }
}
```

Hình 9. Code minh họa Priority Scheduling

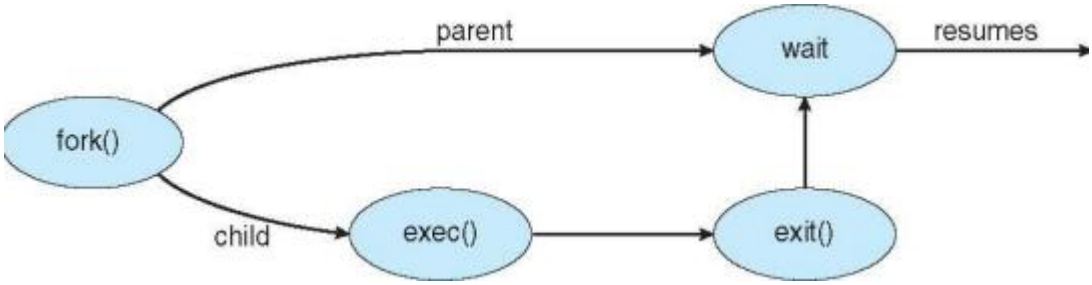
PHẦN 4: THAO TÁC TIẾN TRÌNH & KIẾN TRÚC ĐA TIẾN TRÌNH

4.1 Cơ chế tạo tiến trình cha – con

Trong hệ điều hành, một tiến trình có thể tạo ra các tiến trình mới để thực hiện các nhiệm vụ song song.

fork(): Là lời gọi hệ thống (system call) đặc trưng của UNIX/Linux. Nó tạo ra một bản sao chính xác của tiến trình cha (gọi là tiến trình con). Sau khi fork(), cả hai tiến trình tiếp tục chạy từ cùng một điểm, nhưng có ID tiến trình (PID) khác nhau.

exec(): Thường được dùng sau fork(). Lệnh này thay thế toàn bộ không gian bộ nhớ của tiến trình hiện tại bằng một chương trình mới.



Hình 10. Sơ đồ chu trình tạo tiến trình bằng lệnh fork() và exec()

So sánh UNIX và Windows:

Đặc điểm	UNIX	Windows
Hàm hệ thống	Sử dụng tổ hợp fork() và exec().	Sử dụng hàm duy nhất là CreateProcess().
Không gian địa chỉ	Tiến trình con được tạo ra như một bản sao của cha trước khi nạp mã mới.	Tiến trình con được tạo ra với một không gian địa chỉ mới và nạp mã ngay từ đầu.
Kế thừa tài nguyên	Mặc định kế thừa tất cả các bản tin (file descriptors) và biến môi trường từ cha.	Phải cấu hình các cờ (flags) và tham số để quyết định tài nguyên nào được kế thừa.

Sự linh hoạt	Rất cao: Cha có thể thay đổi thiết lập của con ngay sau khi fork() nhưng trước khi exec().	Thấp hơn: Mọi thiết lập phải được xác định rõ ràng ngay trong các tham số của CreateProcess().
Tối ưu hóa	Sử dụng kỹ thuật Copy-on-Write để tiết kiệm tài nguyên khi nhân đôi bộ nhớ.	Trực tiếp cấp phát không gian bộ nhớ mới, không cần cơ chế nhân đôi từ cha.

Nhiệm vụ thảo luận

Dựa vào vòng lặp hệ thống và sự khác biệt cốt lõi về không gian địa chỉ (Address Space), sự khác nhau giữa việc tiến trình con chỉ sử dụng fork() so với việc sử dụng thêm lệnh exec() ngay sau đó là gì?

Trong hệ điều hành, fork() và exec() là hai lời gọi hệ thống quan trọng dùng để tạo và quản lý tiến trình.

Khi một tiến trình gọi fork(), hệ điều hành sẽ tạo ra một tiến trình con (child process) gần như giống hệt tiến trình cha. Tiến trình con sẽ có PID riêng, nhưng ban đầu nó sao chép không gian địa chỉ (address space) của tiến trình cha, bao gồm mã chương trình, dữ liệu và stack. Sau khi fork(), cả tiến trình cha và tiến trình con sẽ tiếp tục thực thi từ vị trí lệnh ngay sau lời gọi fork().

Trong khi đó, nếu sau khi fork() tiến trình con gọi thêm exec(), thì toàn bộ không gian địa chỉ của tiến trình con sẽ bị thay thế bằng một chương trình mới. Điều này có nghĩa là tiến trình con sẽ không còn chạy chương trình ban đầu của tiến trình cha nữa mà sẽ thực thi một chương trình hoàn toàn khác.

Do đó:

- Chỉ dùng fork(): tiến trình con là bản sao của tiến trình cha và chạy cùng chương trình.
- fork() + exec(): tiến trình con được tạo ra trước, sau đó nạp và chạy một chương trình mới.

Cơ chế fork() kết hợp với exec() thường được sử dụng trong các hệ điều hành như Linux để tạo tiến trình mới, ví dụ khi người dùng chạy một lệnh trong shell.

Nhiệm vụ thảo luận

Giả lập thực thi đoạn code UNIX: Trình bày từng bước giá trị trả về của biến pid. Điều gì sẽ xảy ra trong terminal nếu lập trình viên quên gọi hàm wait(NULL) ở tiến trình cha.

1. Giả lập quá trình thực thi chương trình

Đoạn chương trình UNIX sử dụng lời gọi hệ thống fork() để tạo tiến trình con từ tiến trình cha.

```
pid = fork();
if (pid < 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
}
else if (pid == 0) { /* child process */
    execlp("/bin/ls", "ls", NULL);
}
else { /* parent process */
    wait(NULL);
    printf("Child Complete");
}
```

Hình 11. Code minh họa quá trình thực thi chương trình

2. Giá trị trả về của biến pid

Lời gọi fork() có ba trường hợp giá trị trả về:

Giá trị pid	Ý nghĩa	Tiến trình nhận
pid < 0	Lỗi khi tạo tiến trình	Chỉ xảy ra khi hệ điều hành không thể tạo tiến trình mới
pid = 0	Đang ở tiến trình con	Child process
pid > 0	PID của tiến trình con	Parent process

2.1 Trình tự thực thi giả lập

Giả sử tiến trình ban đầu có PID = **1000**.

Bước 1 – Tiến trình cha gọi fork()

- Hệ điều hành tạo tiến trình con với PID mới (ví dụ: **1001**).

Bước 2 – Sau khi fork() hoàn thành

Tiến trình	Giá trị pid
Parent process (PID 1000)	pid = 1001
Child process (PID 1001)	pid = 0

2.2 Nhánh thực thi của từng tiến trình

Trường hợp 1: pid < 0

Nếu hệ điều hành không thể tạo tiến trình mới: **Fork Failed**

Trường hợp 2: pid == 0 (Tiến trình con)

Tiến trình con thực hiện: **execlp("/bin/ls", "ls", NULL);**

Hàm exec() **thay thế không gian địa chỉ của tiến trình con bằng chương trình mới**. Trong ví dụ này, tiến trình con chạy lệnh: **ls**

- ➢ Terminal hiển thị danh sách file trong thư mục hiện tại.

Theo lý thuyết hệ điều hành, exec() dùng để **thay thế chương trình hiện tại của tiến trình bằng một chương trình khác**.

Trường hợp 3: pid > 0 (Tiến trình cha)

Tiến trình cha thực hiện:

```
wait(NULL);
printf("Child Complete");
```

wait(NULL) khiến tiến trình cha **chờ tiến trình con kết thúc trước khi tiếp tục**.

Sau khi tiến trình con hoàn thành: **Child Complete**

3. Kết quả hiển thị trên Terminal (trường hợp bình thường)

Giả sử thư mục chứa các file:

main.c

a.out

test.txt

Khi chương trình chạy, terminal sẽ hiển thị:

main.c

a.out

test.txt

Child Complete

Giải thích:

1. Tiến trình con chạy lệnh **ls**
2. Tiến trình cha chờ bằng **wait()**
3. Khi tiến trình con kết thúc tiến trình cha in **"Child Complete"**

4. Điều gì xảy ra nếu bỏ wait(NULL)?

Nếu lập trình viên **quên gọi wait(NULL)**, chương trình trở thành:

```
else {
   printf("Child Complete");
}
```

4.1 Hành vi xảy ra

Khi đó:

- Tiến trình cha **không chờ tiến trình con**
- Hai tiến trình chạy **song song (concurrent)**

Do đó thứ tự output **không còn đảm bảo**.

4.2 Các khả năng hiển thị trên terminal

Trường hợp 1:

Child Complete
main.c
a.out
test.txt

Trường hợp 2:

main.c
a.out
test.txt
Child Complete

Trường hợp 3 (hiếm hơn):

Output bị xen kẽ.

4.3 Hiện tượng Zombie Process

Nếu tiến trình con kết thúc trước nhưng tiến trình cha **không gọi wait()**, tiến trình con sẽ trở thành **Zombie Process**.

Zombie process là tiến trình:

- Đã kết thúc thực thi
- Nhưng vẫn tồn tại trong bảng tiến trình

- Chờ tiến trình cha đọc trạng thái kết thúc bằng **wait()**

Nếu nhiều zombie tích lũy có thể gây lãng phí tài nguyên hệ thống.

5. Kết luận

Từ phân tích trên có thể rút ra các kết luận sau:

1. Lời gọi hệ thống fork() tạo một tiến trình con bằng cách sao chép tiến trình cha.
2. Giá trị trả về của fork() cho phép phân biệt tiến trình cha và tiến trình con.
3. Tiến trình con thường sử dụng exec() để thực thi chương trình khác.
4. Lời gọi wait() giúp tiến trình cha **đồng bộ hóa với tiến trình con** và tránh hiện tượng zombie process.
5. Nếu bỏ wait(), tiến trình cha và con chạy đồng thời, dẫn đến **thứ tự output không xác định** và có nguy cơ xuất hiện zombie process.

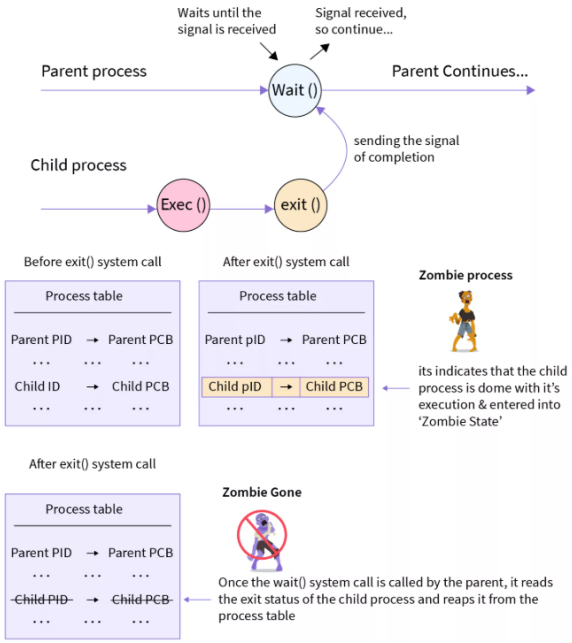
4.2 Tiến trình Zombie và Orphan

Việc quản lý vòng đời tiến trình đôi khi xảy ra các trạng thái "lỗi nhịp":

a. Tiến trình Zombie:

Nguyên nhân: Tiến trình con đã kết thúc nhưng tiến trình cha chưa gọi hàm wait() để đọc trạng thái kết thúc của nó. Khi đó, thông tin của con vẫn nằm trong bảng tiến trình (Process Table).

Xử lý: Hệ điều hành giữ lại một ít tài nguyên để cha có thể đọc. Nếu cha "bỏ quên", các zombie này sẽ chiếm dụng ID tiến trình.

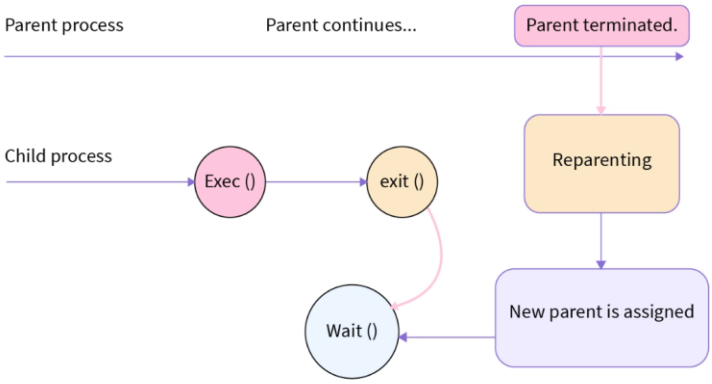


Hình 12. Tiến trình Zombie

b. Tiến trình Orphan:

Nguyên nhân: Tiến trình cha kết thúc trước khi tiến trình con hoàn thành.

Xử lý: Hệ điều hành (thường là tiến trình init hoặc systemd với PID = 1) sẽ "nhận nuôi" các tiến trình mồ côi này và tự động gọi wait() khi chúng kết thúc để dọn dẹp tài nguyên.



Hình 13. Tiến trình Orphan

Nhiệm vụ thảo luận

Trình bày rủi ro tài nguyên đối với hệ thống nếu có quá nhiều tiến trình Zombie xuất hiện cùng lúc. Trình bày cách Hệ Điều Hành chủ động chủ động xử lý và 'nhận nuôi' một tiến trình Orphan như thế nào?

1. Rủi ro tài nguyên đối với hệ thống nếu có quá nhiều Zombie process xuất hiện cùng lúc

1.1 Bản chất của rủi ro

Zombie process không tiếp tục tiêu thụ CPU để tính toán như tiến trình đang chạy, nhưng chúng vẫn chiếm giữ **tài nguyên quản lý của nhân hệ điều hành**, cụ thể là:

- Mã định danh tiến trình (PID),
- Mục nhập trong bảng tiến trình,
- Một phần thông tin trạng thái kết thúc để tiến trình cha truy hồi,
- Cấu trúc điều khiển tiến trình liên quan đến quản lý tiến trình.

1.2 Các rủi ro chính đối với hệ thống

Thứ nhất, cạn kiệt bảng tiến trình hoặc PID khả dụng.

Mỗi zombie vẫn giữ lại một mục trong cơ chế quản lý tiến trình. Nếu số lượng zombie tăng lớn, hệ điều hành có thể thiếu chỗ để ghi nhận các tiến trình mới. Khi đó, các lời gọi tạo tiến trình như fork() có thể thất bại hoặc hệ thống giảm mạnh khả năng tiếp nhận tác vụ mới. Điều này đặc biệt nguy hiểm trên các hệ thống máy chủ sinh tiến trình thường xuyên.

Thứ hai, suy giảm hiệu quả quản trị tiến trình.

Hệ điều hành phải duy trì và quét qua danh sách tiến trình để lập lịch, theo dõi quan hệ cha–con, thu hồi tài nguyên và xử lý trạng thái. Khi có quá nhiều zombie, chi phí quản lý nội bộ tăng lên, gây ảnh hưởng gián tiếp tới hiệu năng chung của hệ thống.

Thứ ba, che giấu lỗi lập trình hoặc lỗi đồng bộ hóa trong ứng dụng.

Sự xuất hiện hàng loạt zombie thường phản ánh việc tiến trình cha không gọi wait() đúng cách. Đây là dấu hiệu của thiết kế tiến trình kém, thiếu đồng bộ hóa giữa cha và con.

Thứ tư, ảnh hưởng đến tính ổn định lâu dài của hệ thống.

Trong ngắn hạn, một vài zombie có thể không gây hậu quả nghiêm trọng. Tuy nhiên, nếu zombie tích lũy liên tục trong môi trường chạy lâu dài, hệ thống có thể rơi vào trạng thái suy giảm tài nguyên quản trị, khó tạo thêm tiến trình mới, từ đó làm giảm độ tin cậy và tính sẵn sàng của dịch vụ.

1.3 Đánh giá tổng quát

Vì zombie process là tiến trình đã kết thúc nhưng chưa được "thu hoạch" trạng thái, rủi ro lớn nhất không nằm ở CPU mà nằm ở **sự chiếm dụng cấu trúc quản lý tiến trình của hệ điều hành**. Do đó, càng nhiều zombie xuất hiện đồng thời thì nguy cơ **nghẽn tài nguyên quản lý tiến trình** càng cao.

2. Hệ điều hành chủ động xử lý và "nhận nuôi" tiến trình Orphan như thế nào?

2.1 Bản chất của orphan process

Khác với zombie, orphan process **vẫn còn đang thực thi**. Vì tiến trình cha không còn nữa nên hệ điều hành phải tái tổ chức quan hệ cha–con để tiến trình này không bị mất cơ chế quản lý.

Trong hệ UNIX/Linux, cơ chế phổ biến là **re-parenting**: tiến trình mồ côi được gán lại cho một tiến trình hệ thống đặc biệt, thường là init (PID = 1) hoặc một tiến trình quản lý tương đương do hệ điều hành chỉ định. Hình minh họa của đề bài cũng thể hiện rõ orphan process được **re-parented to init (pid=1)**.

2.2 Cách hệ điều hành "nhận nuôi"

Quy trình xử lý có thể trình bày như sau:

Bước 1: Tiến trình cha chấm dứt

Khi tiến trình cha kết thúc trước tiến trình con, quan hệ cha–con ban đầu bị phá vỡ.

Bước 2: Hệ điều hành phát hiện tiến trình con đang còn sống

Vì hệ điều hành luôn quản lý tiến trình qua PID, PCB và quan hệ cây tiến trình, nên nó có thể xác định những tiến trình con nào trở thành mồ côi.

Bước 3: Thực hiện re-parenting

Hệ điều hành gán tiến trình orphan sang một tiến trình cha mới ở mức hệ thống, điển hình là init. Từ thời điểm đó, init trở thành tiến trình cha hợp pháp của orphan.

Bước 4: Thu hồi trạng thái khi tiến trình con kết thúc

Khi orphan kết thúc trong tương lai, tiến trình cha mới (init) sẽ gọi cơ chế tương tự wait() để nhận trạng thái kết thúc và giải phóng hoàn toàn thông tin quản lý còn sót lại. Nhờ vậy, orphan không tiếp tục biến thành zombie tồn tại lâu dài.

2.3 Ý nghĩa của cơ chế "nhận nuôi"

Cơ chế này có ba ý nghĩa quan trọng:

- **Bảo đảm mọi tiến trình đang chạy đều thuộc phạm vi quản lý của hệ điều hành.**

- **Ngăn tiến trình mồ côi trở thành đối tượng không kiểm soát.**
- **Đảm bảo khi orphan kết thúc thì vẫn có một tiến trình cha hợp lệ thu hồi trạng thái của nó.**

Nói cách khác, cơ chế "nhận nuôi" là biện pháp giúp hệ điều hành duy trì tính toàn vẹn của cây tiến trình và tránh rò rỉ thông tin quản lý tiến trình.

3. Liên hệ giữa Zombie và Orphan

Hai trạng thái này đều phát sinh từ quan hệ cha–con trong quản lý tiến trình, nhưng bản chất khác nhau:

- **Zombie**: tiến trình con **đã kết thúc**, nhưng cha chưa gọi wait().
- **Orphan**: tiến trình con **chưa kết thúc**, nhưng cha đã chấm dứt.

Do đó:

- Zombie gây áp lực lên **bảng tiến trình và tài nguyên quản lý**.
- Orphan được hệ điều hành **tái gán cha** để tiếp tục được quản lý hợp lệ.

4. Kết luận

Zombie process là dấu hiệu cho thấy tiến trình cha không thu hồi trạng thái kết thúc của tiến trình con bằng wait(). Nếu có quá nhiều zombie cùng lúc, hệ thống có thể bị chiếm dụng bảng tiến trình, giảm khả năng tạo tiến trình mới và suy giảm hiệu năng quản trị. Trong khi đó, orphan process không bị bỏ mặc hoàn toàn; hệ điều hành sẽ chủ động **re-parent** tiến trình này về một tiến trình hệ thống như init để tiếp tục quản lý và thu hồi trạng thái khi nó kết thúc. Cơ chế này thể hiện vai trò trung tâm của hệ điều hành trong việc duy trì tính ổn định, nhất quán và an toàn của cây tiến trình.

4.3 Nghiên cứu điển hình: Google Chrome

Trước đây, các trình duyệt web thường sử dụng kiến trúc đơn tiến trình (Single-process). Tuy nhiên, Google Chrome đã tạo ra bước ngoặt khi áp dụng mô hình đa tiến trình (Multi-

process Architecture), coi mỗi thành phần của trình duyệt như một thực thể độc lập trong hệ điều hành.

a. Các loại tiến trình cốt lõi

Chrome phân chia nhiệm vụ cho các loại tiến trình khác nhau để đảm bảo tính chuyên hóa:

Browser Process (*Tiến trình trình duyệt*): Là tiến trình cha, đóng vai trò điều phối. Nó quản lý giao diện người dùng (thanh địa chỉ, các nút điều hướng), quản lý các tiến trình con và thực hiện các thao tác đặc quyền như truy cập mạng hoặc hệ thống tệp.

Renderer Process (*Tiến trình kết xuất*): Thông thường, mỗi tab hoặc mỗi trang web sẽ được chạy trong một Renderer Process riêng biệt. Tiến trình này chịu trách nhiệm chuyển đổi mã HTML, CSS, JavaScript thành giao diện hình ảnh mà người dùng nhìn thấy.

Plugin Process (*Tiến trình tiện ích*): Quản lý các trình cắm (như Flash trước đây hoặc các extension). Việc tách riêng giúp đảm bảo rằng nếu một plugin bị lỗi, nó sẽ không gây ảnh hưởng đến trang web đang hiển thị.

GPU Process: Chuyên trách xử lý các tác vụ đồ họa và tăng tốc phần cứng, giúp việc hiển thị các nội dung đa phương tiện mượt mà hơn mà không làm nặng CPU chính.

b. Lợi ích về độ ổn định (Stability)

Trong cơ chế đơn tiến trình, nếu một tab gặp lỗi (ví dụ: lỗi vòng lặp vô tận trong JavaScript), toàn bộ trình duyệt sẽ bị treo. Với kiến trúc của Chrome:

Khả năng cô lập lỗi (Fault Isolation): Do mỗi tab là một tiến trình Renderer riêng biệt, nếu tab đó bị crash, hệ điều hành chỉ việc đóng tiến trình đó lại. Các tab khác và tiến trình Browser chính vẫn hoạt động bình thường.

Quản lý tài nguyên: Hệ điều hành có thể nhận diện chính xác tab nào đang tiêu tốn quá nhiều RAM hoặc CPU để người dùng dễ dàng tắt bỏ thông qua "Task Manager" riêng của Chrome.

c. Lợi ích về bảo mật (Security & Sandboxing)

Đây là ưu điểm quan trọng nhất trong môn Hệ điều hành khi nói về Chrome:

Cơ chế Sandbox (Hố cát): Renderer Process (nơi thực thi mã từ internet) thường xuyên phải đối mặt với nguy cơ chứa mã độc. Chrome áp dụng cơ chế Sandbox để "nhốt" các tiến trình này lại. Một Renderer Process bị giới hạn quyền truy cập vào các tài nguyên nhạy cảm như ổ cứng hoặc camera.

Ngăn chặn leo thang đặc quyền: Nếu hacker tấn công thành công một tab, chúng chỉ chiếm được quyền kiểm soát bên trong "hố cát" đó mà không thể xâm nhập vào hệ thống tệp của máy tính hoặc đánh cắp dữ liệu từ các tab ngân hàng đang mở bên cạnh. Mọi yêu cầu truy cập tài nguyên hệ thống từ Renderer đều phải thông qua Browser Process để kiểm tra quyền hạn.

Nhiệm vụ thảo luận

Vì sao Google Chrome chọn kiến trúc đa tiến trình thay vì đơn tiến trình?

1. Lý do Chrome không chọn kiến trúc đơn tiến trình

Về mặt lý thuyết, mô hình đơn tiến trình có ưu điểm là **tiết kiệm bộ nhớ hơn**, do mọi tab và thành phần cùng chia sẻ một không gian địa chỉ và các cấu trúc quản lý. Tuy nhiên, đổi lại, mô hình này tồn tại rủi ro rất lớn: **mọi thành phần phụ thuộc chặt vào cùng một tiến trình**, nên lỗi tại một tab, một đoạn JavaScript hoặc một plug-in có thể lan ra toàn bộ trình duyệt. Từ góc nhìn hệ điều hành, khi các chức năng cùng nằm trong một tiến trình, mức độ cô lập thấp; sự cố ở một phần dễ ảnh hưởng đến các phần còn lại. Điều này trái với yêu cầu của trình duyệt hiện đại, nơi người dùng đồng thời mở nhiều trang web, chạy nhiều mã JavaScript và tương tác với nhiều tài nguyên mạng, đồ họa, I/O.

Do đó, Chrome chấp nhận **chi phí bộ nhớ lớn hơn** để đổi lấy **độ tin cậy, tính an toàn và khả năng kiểm soát lỗi** cao hơn. Đây là một quyết định thiết kế mang tính hệ thống: hy sinh một phần hiệu quả tài nguyên để tối ưu cho **ổn định và bảo mật**.

2. Phân tích quyết định dưới góc độ Sandboxing (hộp cát bảo mật)

2.1 Sandbox là gì trong bối cảnh này?

Sandbox có thể hiểu là một **môi trường thực thi bị giới hạn quyền hạn**. Tiến trình renderer vẫn có thể xử lý HTML, CSS, JavaScript và hiển thị nội dung web, nhưng không được tự do truy cập toàn bộ tài nguyên hệ thống như một tiến trình thông thường.

2.2 Vì sao đa tiến trình giúp sandbox hiệu quả hơn?

Sandbox chỉ thực sự hiệu quả khi đối tượng bị kiểm soát được **tách riêng thành tiến trình độc lập**. Nếu toàn bộ trình duyệt chỉ là một tiến trình duy nhất, thì rất khó giới hạn quyền một phần chức năng mà không làm ảnh hưởng tới toàn bộ chương trình. Nhưng khi Chrome tách renderer thành các tiến trình riêng, hệ điều hành có thể áp đặt chính sách hạn chế trực tiếp lên từng tiến trình đó.

Nói cách khác:

- **Browser process** giữ các quyền quan trọng hơn để quản lý giao diện, tập tin, thiết bị và I/O.
- **Renderer process** chỉ đảm nhận việc hiển thị nội dung web và bị hạn chế đặc quyền.
- **Plug-in process** được tách biệt để nếu có rủi ro thì không lan thẳng vào phần lõi trình duyệt.

2.3 Lợi ích bảo mật của sandboxing

Thiết kế này giúp giảm thiểu hậu quả nếu trang web độc hại khai thác lỗ hổng trong bộ máy dựng trang. Khi renderer bị tấn công, mã độc vẫn bị "nhốt" trong phạm vi quyền hạn thấp, khó truy cập trực tiếp vào hệ thống tệp, thiết bị hoặc phần điều khiển chính của trình duyệt. Vì vậy, **đa tiến trình không chỉ là giải pháp tổ chức chức năng**, mà còn là **cơ chế phân quyền bảo mật dựa trên tiến trình**.

3. Phân tích dưới góc độ quản lý lỗi (fault isolation)

3.1 Cô lập lỗi theo từng tab

Khi mỗi tab hoặc website chạy trên renderer process riêng, lỗi phát sinh trong tab A sẽ chủ yếu ảnh hưởng đến tab A, thay vì làm sập toàn bộ phiên duyệt web. Đây là tư duy "chia

nhỏ miền hỏng hóc” trong thiết kế hệ thống: thay vì một lỗi làm hỏng toàn cục, lỗi chỉ tác động cục bộ.

3.2 Cô lập lỗi của plug-in và tiện ích mở rộng

Plug-in hoặc extension thường là nguồn gây mất ổn định vì chúng có thể do bên thứ ba phát triển. Khi được đặt trong tiến trình tách biệt, lỗi của chúng ít khả năng làm hỏng browser process hoặc các tab khác. Cơ chế này giúp Chrome vận hành bền vững hơn trong môi trường thực tế, nơi thành phần mở rộng rất đa dạng và khó kiểm soát tuyệt đối.

3.3 Liên hệ với lý thuyết tiến trình của hệ điều hành

Hệ điều hành quản lý tiến trình thông qua PCB, trạng thái, thông tin bộ nhớ, I/O và lập lịch; đồng thời các tiến trình có thể là độc lập hoặc cộng tác với nhau qua cơ chế IPC. Điều này rất phù hợp với Chrome: mỗi thành phần chính được tách thành tiến trình riêng để có trạng thái, không gian thực thi và mức quyền riêng; còn việc phối hợp giữa browser process với renderer process được thực hiện theo mô hình cộng tác giữa các tiến trình.

Như vậy, về bản chất, Chrome tận dụng chính cơ chế nền tảng của hệ điều hành: **tách tiến trình để tăng cô lập**, sau đó **dùng cơ chế phối hợp liên tiến trình để vẫn giữ được tính thống nhất của ứng dụng**.

4. Vì sao dù tốn nhiều bộ nhớ hơn vẫn là lựa chọn hợp lý?

Kiến trúc đa tiến trình làm tăng tiêu thụ bộ nhớ vì:

- Mỗi tiến trình cần không gian địa chỉ riêng;
- Mỗi tiến trình cần cấu trúc quản lý riêng trong hệ điều hành;
- Chuyển đổi giữa các tiến trình phát sinh thêm chi phí quản lý;
- Giao tiếp giữa các thành phần phải thông qua cơ chế IPC thay vì chỉ gọi hàm nội bộ.

Theo lý thuyết hệ điều hành, chuyển ngữ cảnh giữa các tiến trình là một **chi phí quản lý**, và hệ điều hành phải lưu/khôi phục ngữ cảnh tiến trình khi chuyển CPU từ tiến trình này sang tiến trình khác.

Tiến trình nói chung cũng "nặng" hơn tiểu trình, vì một tiến trình có không gian tài nguyên và cơ chế quản lý độc lập; ngược lại, tiểu trình tiết kiệm hơn do chia sẻ tài nguyên trong cùng tiến trình.

Tuy nhiên, đối với Chrome, **chi phí bộ nhớ tăng là cái giá chấp nhận được** vì nó đổi lại bốn lợi ích lớn:

- tăng độ ổn định của toàn trình duyệt;
- giảm phạm vi ảnh hưởng khi một tab hoặc plug-in lỗi;
- tăng mức độ an toàn nhờ sandboxing;
- cải thiện trải nghiệm người dùng vì không phải mất toàn bộ phiên làm việc chỉ do một trang gặp sự cố.

Từ góc nhìn kỹ thuật hệ điều hành, đây là bài toán **trade-off**: **đơn tiến trình tối ưu tài nguyên hơn**, nhưng **đa tiến trình tối ưu độ tin cậy và an toàn hơn**. Với một trình duyệt web hiện đại, nhất là trình duyệt phải xử lý mã từ Internet không đáng tin cậy, Chrome đã ưu tiên đúng tiêu chí quan trọng hơn.

5. Kết luận

Google Chrome lựa chọn kiến trúc **đa tiến trình** thay vì **đơn tiến trình** không phải vì tiết kiệm tài nguyên, mà vì mục tiêu cốt lõi là **an toàn, ổn định và dễ kiểm soát lỗi**. Trong mô hình này, browser process, renderer process và plug-in process được tách biệt rõ ràng; đặc biệt renderer chạy trong **sandbox**, bị hạn chế quyền truy cập đĩa và I/O, nhờ đó giảm thiểu nguy cơ khai thác bảo mật. Đồng thời, khi mỗi tab hoặc website được xử lý trong tiến trình riêng, lỗi ở một tab khó làm sập toàn bộ trình duyệt. Dù thiết kế này tiêu tốn nhiều bộ nhớ hơn và làm tăng chi phí quản lý tiến trình, nó vẫn là quyết định hợp lý vì đáp ứng tốt hơn

yêu cầu của trình duyệt hiện đại: **bảo mật cao, cô lập lỗi tốt và trải nghiệm sử dụng ổn định**.

PHẦN 5: TRUYỀN THÔNG LIÊN TIẾN TRÌNH (IPC)

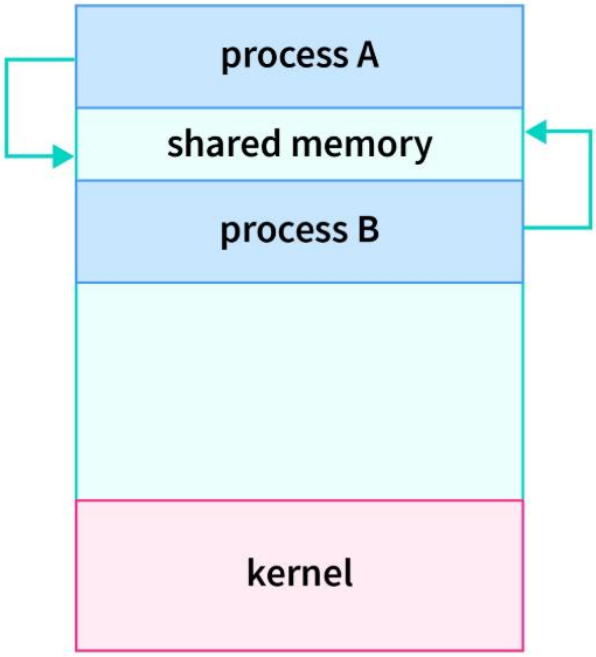
5.1 Lý do cần IPC

Các tiến trình thường cần hợp tác để chia sẻ dữ liệu, tăng tốc độ tính toán hoặc module hóa hệ thống. Tuy nhiên, vì mỗi tiến trình có không gian địa chỉ độc lập, chúng cần một cơ chế trung gian để "nói chuyện" với nhau.

5.2 Mô hình bộ nhớ chia sẻ (Shared Memory)

a. Cơ chế hoạt động

Trong một hệ thống bình thường, hệ điều hành ngăn cản các tiến trình truy cập vào bộ nhớ của nhau để bảo mật. Tuy nhiên, với Shared Memory, hệ điều hành sẽ "phá lệ" thông qua các bước:



Hình 14. Mô hình bộ nhớ chia sẻ (Shared Memory)

- Tạo vùng nhớ (Creation): Một tiến trình yêu cầu hệ điều hành tạo ra một phân đoạn bộ nhớ dùng chung.
- Gắn kết (Attachment): Các tiến trình khác muốn tham gia giao tiếp phải thực hiện lệnh "attach" để ánh xạ vùng nhớ vật lý đó vào không gian địa chỉ ảo của chính nó.
- Trao đổi: Dữ liệu được ghi bởi tiến trình này sẽ ngay lập tức xuất hiện trong không gian của tiến trình kia mà không cần thông qua các hàm hệ thống trung gian.

b. Ưu điểm và hạn chế

Về tốc độ: Đây là phương thức IPC có hiệu năng cao nhất. Vì dữ liệu nằm ngay trong bộ nhớ của tiến trình, tốc độ truy cập tương đương với tốc độ truy xuất RAM cục bộ, không bị trễ do chuyển đổi ngữ cảnh (Context Switch) hay sao chép dữ liệu qua Nhân (Kernel).

Về khả năng mở rộng: Rất hiệu quả khi cần chia sẻ các cấu trúc dữ liệu khổng lồ (như cơ sở dữ liệu hoặc các mảng tính toán đồ họa) mà việc gửi thông điệp thông thường sẽ gây nghẽn hệ thống.

Hạn chế: Khó triển khai trong hệ thống phân tán (nhiều máy tính khác nhau) vì bộ nhớ không thể chia sẻ vật lý qua mạng.

c. Vấn đề đồng bộ và Bài toán Producer – Consumer

Đây là thách thức lớn nhất của mô hình này. Khi hai tiến trình không "biết" người kia đang làm gì, chúng có thể ghi đè lên nhau.

Mô hình Producer – Consumer: * Producer (Người sản xuất): Tạo ra thông tin và đặt vào vùng nhớ đệm (buffer) dùng chung.

- Consumer (Người tiêu dùng): Lấy thông tin từ vùng nhớ đệm đó để xử lý.

Điều kiện ràng buộc: * Producer không được ghi thêm nếu buffer đã đầy.

- Consumer không được lấy ra nếu buffer đang trống.

Công cụ điều phối: Để giải quyết, hệ điều hành cung cấp:

- Semaphore: Một biến đếm dùng để báo hiệu trạng thái của buffer (còn bao nhiêu chỗ trống, có bao nhiêu dữ liệu).
- Mutex (Mutual Exclusion): Một cái "khóa" đảm bảo rằng tại một thời điểm, chỉ có DUY NHẤT một tiến trình được quyền can thiệp vào buffer để tránh làm hỏng dữ liệu.

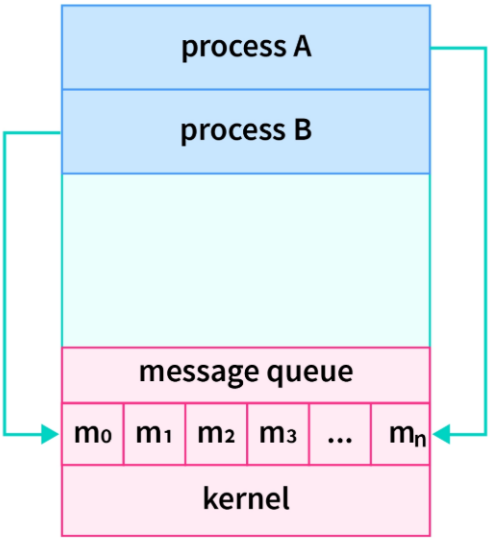
5.3 Mô hình truyền thông thông điệp (Message Passing)

Trong mô hình này, các tiến trình giao tiếp với nhau mà không cần chia sẻ chung một không gian địa chỉ. Thay vào đó, hệ điều hành cung cấp một cơ chế để các tiến trình trao đổi thông tin thông qua các gói tin (messages).

a. Hai thao tác cơ bản

Mọi hệ thống Message Passing đều dựa trên hai lời gọi hệ thống cốt lõi:

- send(message): Gửi một thông điệp đi.
- receive(message): Nhận một thông điệp đến.



Hình 15. Mô hình truyền thông điệp (Message Passing)

b. Cơ chế liên lạc: Trực tiếp và Gián tiếp

Liên lạc trực tiếp (Direct Communication):

- Mỗi tiến trình muốn giao tiếp phải chỉ định rõ tên tiến trình nhận hoặc gửi.
- Ví dụ: send(P, message) gửi tin cho tiến trình P.
- Đặc điểm: Một liên kết (link) được thiết lập tự động giữa hai tiến trình. Tuy nhiên, nếu tên tiến trình thay đổi, mã nguồn phải được cập nhật lại, gây khó khăn cho việc lập trình linh hoạt.

Liên lạc gián tiếp (Indirect Communication):

- Tin nhắn được gửi và nhận thông qua các đối tượng trung gian gọi là Hộp thư (Mailboxes) hoặc Cổng (Ports).
- Ví dụ: send(A, message) gửi vào hộp thư A.
- Ưu điểm: Cho phép nhiều tiến trình cùng chia sẻ một hộp thư. Một tiến trình có thể nhận tin từ nhiều nguồn khác nhau miễn là chúng dùng chung hộp thư đó.

c. Vấn đề đồng bộ hóa (Synchronization)

Tùy vào nhu cầu về hiệu suất, việc gửi và nhận tin có thể được cấu hình theo hai dạng:

Blocking (Đồng bộ - Synchronous):

- Blocking send: Tiến trình gửi sẽ bị tạm dừng (ngủ) cho đến khi tin nhắn được nhận bởi tiến trình đích hoặc bởi hộp thư.
- Blocking receive: Tiến trình nhận sẽ đợi cho đến khi có tin nhắn gửi tới.

Non-blocking (Bất đồng bộ - Asynchronous):

- Non-blocking send: Tiến trình gửi gửi tin xong và tiếp tục thực hiện công việc khác ngay lập tức.
- Non-blocking receive: Tiến trình nhận lấy về một tin nhắn có sẵn hoặc một thông báo "không có tin nhắn nào" (null) để tiếp tục xử lý công việc khác thay vì phải chờ đợi.

Nhiệm vụ thảo luận

Message Passing hay Shared Memory hiệu quả hơn khi chia sẻ dữ liệu rất lớn?

Trong bài toán giả định hai tiến trình cần chia sẻ một khối dữ liệu rất lớn, chẳng hạn dữ liệu phục vụ render một video 4K, **mô hình Shared Memory thường cho hiệu suất băng thông cao hơn Message Passing**. Cơ sở của kết luận này đến từ cách tổ chức truyền thông của hai mô hình IPC trong hệ điều hành.

1. Vì sao Shared Memory có băng thông cao hơn?

Lý do quan trọng nhất là **shared memory tránh được chi phí sao chép dữ liệu lặp lại qua kernel hoặc hàng đợi thông điệp**. Khi hai tiến trình cùng ánh xạ đến một vùng nhớ chung, dữ liệu lớn chỉ cần được đặt trong vùng nhớ đó, sau đó cả tiến trình sản xuất và tiến trình tiêu thụ cùng truy cập trực tiếp. Điều này đặc biệt phù hợp với khối dữ liệu cỡ lớn như frame video 4K, nơi chi phí copy dữ liệu nhiều lần sẽ rất đắt.

Ngược lại, ở mô hình **message passing**, việc trao đổi dựa trên cơ chế gửi và nhận thông điệp; vì vậy dữ liệu phải đi qua tầng truyền thông do hệ điều hành quản lý.

Điều này cho thấy message passing có thêm chi phí trung gian của hệ điều hành, còn shared memory giảm bớt sự can thiệp đó trong pha truyền dữ liệu.

2. Phân tích theo góc độ chi phí hệ thống

Với dữ liệu nhỏ, chênh lệch hiệu năng giữa hai mô hình có thể chưa quá lớn. Tuy nhiên, khi dữ liệu tăng lên rất lớn, message passing bắt đầu bộc lộ bất lợi vì:

- phải đóng gói thông điệp,
- phải gửi qua liên kết truyền thông,
- có thể phát sinh sao chép dữ liệu giữa không gian người dùng và không gian hệ thống,
- có thêm chi phí quản lý hàng đợi thông điệp hoặc mailbox.

Trong khi đó, shared memory chỉ cần:

- thiết lập vùng nhớ chung,
- ghi dữ liệu vào vùng nhớ,
- để tiến trình còn lại đọc trực tiếp từ cùng vùng nhớ đó,
- xử lý đồng bộ hóa để tránh xung đột truy cập.

Chính vì vậy, đối với **mục tiêu tối đa hóa băng thông truyền dữ liệu**, shared memory có ưu thế rõ ràng hơn.

Điều này củng cố nhận định rằng cơ chế truyền thông qua thông điệp tuy an toàn và mô-đun hóa tốt, nhưng không tối ưu cho truyền tải khối dữ liệu rất lớn.

3. Điểm mạnh và điểm yếu của từng mô hình

Shared Memory

Ưu điểm:

- băng thông cao hơn khi xử lý dữ liệu lớn;
- giảm số lần sao chép dữ liệu;
- phù hợp với các bài toán producer-consumer khối lượng lớn.

Nhược điểm:

- khó lập trình hơn;
- phải tự giải quyết đồng bộ hóa;
- dễ phát sinh race condition nếu truy cập đồng thời không được kiểm soát.

Message Passing

Ưu điểm:

- cấu trúc rõ ràng, dễ kiểm soát hơn;
- không cần chia sẻ biến trực tiếp giữa các tiến trình;
- phù hợp hơn với hệ phân tán hoặc khi cần tách biệt tiến trình.

Nhược điểm:

- chi phí truyền dữ liệu cao hơn với dữ liệu lớn;
- băng thông thực tế thấp hơn trong các bài toán multimedia, đồ họa, render lớn;
- phụ thuộc nhiều hơn vào sự tham gia của hệ điều hành trong truyền thông.

4. Kết luận

Đối với bài toán hai tiến trình cần chia sẻ **khối dữ liệu cực lớn** như dữ liệu render video 4K, **Shared Memory là mô hình cho hiệu suất băng thông cao hơn Message Passing**. Nguyên nhân là shared memory cho phép hai tiến trình truy cập trực tiếp cùng một vùng dữ liệu, giảm chi phí sao chép và giảm sự can thiệp của hệ điều hành trong quá trình trao đổi dữ liệu. Ngược lại, message passing phù hợp hơn khi ưu tiên tính mô-đun, tính an toàn giao tiếp và dễ quản lý, nhưng không phải lựa chọn tối ưu nếu mục tiêu chính là **thông lượng truyền dữ liệu lớn với độ trễ thấp**.

Tóm lại:

Nếu ưu tiên **băng thông và hiệu năng** cho dữ liệu lớn → chọn **Shared Memory**. Nếu ưu tiên **đơn giản hóa giao tiếp và cô lập tiến trình** → chọn **Message Passing**.

5.4 So sánh hai mô hình IPC

Tiêu chí	Shared Memory	Message Passing
Hiệu năng dữ liệu lớn	Rất tốt; tránh sao chép lặp lại.	Thường kém hơn do đóng gói và sao chép bản tin.
Đồng bộ hóa	Khó hơn; lập trình viên phải chủ động kiểm soát truy cập.	Đơn giản hơn ở mức chia sẻ dữ liệu vì giao tiếp qua bản tin.
Cô lập lỗi dữ liệu	Thấp hơn nếu vùng nhớ dùng chung bị ghi sai.	Cao hơn vì các tiến trình không trực tiếp chia sẻ vùng nhớ.
Khả năng mở rộng qua mạng	Hạn chế, chủ yếu hiệu quả trong cùng máy.	Tốt hơn, phù hợp cho hệ phân tán và kiến trúc dịch vụ.
Độ phức tạp triển khai	Cao ở phần đồng bộ và vòng đời vùng nhớ.	Cao ở phần giao thức, hàng đợi và xử lý bất đồng bộ.
Bài toán điển hình	Render video, xử lý ảnh, vùng đệm lớn.	Chat, hàng đợi công việc, dịch vụ phân tán, control plane.

Nhiệm vụ thảo luận

Tư vấn lựa chọn mô hình IPC cho hai hệ thống

1. Hệ thống Chat nội bộ (gửi text ngắn gọn liên tục)

Đối với **hệ thống chat nội bộ**, mô hình phù hợp hơn là **Trao đổi thông điệp (Message Passing)**.

Lý do lựa chọn

Thứ nhất, dữ liệu của hệ thống chat thường là **các bản tin ngắn, độc lập, kích thước nhỏ**, ví dụ tin nhắn văn bản, trạng thái gửi/nhận, thông báo người dùng online hoặc offline. Với loại dữ liệu này, message passing rất phù hợp vì bản chất của nó là gửi và nhận các thông điệp rời rạc giữa các tiến trình.

Thứ hai, message passing **dễ tổ chức và an toàn hơn** trong bối cảnh các tiến trình chỉ cần gửi từng gói dữ liệu nhỏ. Không cần cho các tiến trình cùng truy cập vào một vùng nhớ chung, do đó giảm nguy cơ ghi đè dữ liệu hoặc phát sinh race condition. Trong khi đó, shared memory đòi hỏi phải có cơ chế đồng bộ hóa khi nhiều tiến trình cùng truy cập vùng nhớ dùng chung.

Thứ ba, với chat nội bộ, yêu cầu chính thường là **truyền thông rõ ràng, mô-đun hóa tốt và dễ kiểm soát luồng dữ liệu**, hơn là tối đa hóa băng thông cực lớn. Do đó, việc chấp nhận chi phí system call và hàng đợi thông điệp là hợp lý để đổi lấy tính đơn giản trong thiết kế.

Kết luận cho hệ thống chat

Với hệ thống chat nội bộ gửi văn bản ngắn liên tục, **Message Passing là lựa chọn phù hợp hơn** vì:

- phù hợp với dữ liệu nhỏ, dạng bản tin;
- dễ lập trình và kiểm soát;
- an toàn hơn do không phải chia sẻ trực tiếp cùng một vùng nhớ;
- hỗ trợ tự nhiên cho mô hình gửi/nhận tin nhắn giữa các tiến trình.

2. Trình biên tập Video (nhiều tiến trình cùng thao tác trên một mảng pixel chung)

Đối với **trình biên tập video**, mô hình phù hợp hơn là **Bộ nhớ chia sẻ (Shared Memory)**.

Lý do lựa chọn

Thứ nhất, bài toán này liên quan đến **khối dữ liệu rất lớn**: nhiều tiến trình cùng thao tác trên một mảng pixel chung, có thể là từng khung hình video, bộ đệm ảnh, hoặc dữ liệu màu

của hàng triệu điểm ảnh. Nếu dùng message passing, dữ liệu phải được đóng gói thành thông điệp và gửi/nhận nhiều lần, gây chi phí sao chép lớn. Trong khi đó, shared memory cho phép các tiến trình cùng truy cập trực tiếp vào một vùng nhớ chung, nhờ đó giảm đáng kể chi phí truyền dữ liệu.

Thứ hai, **Producer–Consumer** bằng giải pháp shared memory với vùng đệm buffer, chỉ số in, out, và nhấn mạnh đây là cách các tiến trình cùng thao tác trên dữ liệu dùng chung.

Điều này rất gần với mô hình của trình biên tập video: nhiều tiến trình hoặc nhiều đơn vị xử lý cùng làm việc trên cùng một tập dữ liệu lớn.

Thứ ba, với video editor, mục tiêu quan trọng là **tốc độ cao và truy cập liên tục**, ví dụ:

- một tiến trình giải mã frame,
- một tiến trình áp hiệu ứng,
- một tiến trình render preview,
- một tiến trình mã hóa đầu ra.

Trong trường hợp này, shared memory cho băng thông tốt hơn vì dữ liệu không phải chuyển qua lại bằng từng thông điệp riêng lẻ. Điểm đánh đổi là lập trình viên phải tự lo cơ chế đồng bộ hóa để tránh xung đột khi nhiều tiến trình cùng đọc/ghi.

Kết luận cho trình biên tập video

Với trình biên tập video nơi nhiều tiến trình cùng thao tác trên **một mảng pixel chung**, **Shared Memory là lựa chọn phù hợp hơn** vì:

- dữ liệu rất lớn và cần truy cập liên tục;
- giảm chi phí sao chép dữ liệu;
- cho băng thông cao hơn;
- phù hợp với các tác vụ xử lý ảnh/video theo kiểu chia sẻ vùng đệm chung.

3. Nhận xét mở rộng

Ngoài IPC giữa các tiến trình, **tiểu trình (threads)** thường được ưu tiên khi các tác vụ nằm trong cùng một tiến trình, vì các tiểu trình chia sẻ tài nguyên dễ dàng hơn so với việc dùng shared memory hay message passing giữa hai tiến trình khác nhau. Đồng thời, tiểu trình có chi phí tạo và chuyển ngữ cảnh thấp hơn tiến trình.

Điều này gợi ý rằng:

- với **chat nội bộ**, nếu các thành phần tách biệt thành nhiều tiến trình, nên chọn **message passing**;
- với **video editor**, nếu thực sự cần nhiều tiến trình độc lập, nên chọn **shared memory**; còn nếu các tác vụ nằm trong cùng một ứng dụng, có thể cân nhắc **đa tiểu trình** để còn hiệu quả hơn.

4. Kết luận cuối cùng

1. Hệ thống Chat nội bộ: chọn **Message Passing**
Vì dữ liệu là các bản tin nhỏ, rời rạc, cần cơ chế gửi/nhận rõ ràng, dễ kiểm soát và an toàn hơn trong thiết kế.

2. Trình biên tập Video: chọn **Shared Memory**
Vì phải xử lý khối dữ liệu pixel rất lớn, truy cập liên tục, cần băng thông cao và giảm chi phí sao chép dữ liệu giữa các tiến trình.

Tóm tắt ngắn:

- **Dữ liệu nhỏ, phân tán, dạng bản tin → Message Passing**
- **Dữ liệu lớn, truy cập liên tục, hiệu năng cao → Shared Memory**

5.5 PHẦN MINH HỌA/ THỰC HÀNH

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Đoạn chương trình trên minh họa cơ chế tạo tiến trình cha – con bằng system call fork() trong hệ điều hành UNIX/Linux.

Đầu tiên, lời gọi fork() được thực hiện để tạo một tiến trình con. Sau khi gọi fork(), hệ điều hành tạo một bản sao của tiến trình cha, bao gồm không gian địa chỉ, các thanh ghi và thông tin quản lý tiến trình (PCB). Tuy nhiên, mỗi tiến trình sẽ có PID riêng biệt.

Chương trình phân nhánh thành ba trường hợp:

- Nếu pid < 0: quá trình tạo tiến trình thất bại.
- Nếu pid == 0: đây là tiến trình con.
- Nếu pid > 0: đây là tiến trình cha.

Trong tiến trình con, hàm execlp() được gọi để thực thi lệnh ls. Hàm này thay thế toàn bộ nội dung chương trình hiện tại của tiến trình con bằng chương trình mới (lệnh liệt kê danh sách thư mục). Điều này minh họa rõ cơ chế exec() trong UNIX, tức là thay thế tiến trình hiện tại bằng một chương trình khác mà không tạo thêm tiến trình mới.

Trong tiến trình cha, hàm wait(NULL) được sử dụng để đợi tiến trình con kết thúc trước khi tiếp tục thực thi. Cơ chế này đảm bảo sự đồng bộ giữa hai tiến trình và tránh tình trạng Zombie process.

Thông qua ví dụ này, có thể thấy sự phối hợp giữa fork(), exec() và wait() trong việc tạo, thực thi và quản lý vòng đời tiến trình trong hệ điều hành UNIX/Linux.

TỔNG HỢP CÁC NHIỆM VỤ THẢO LUẬN

1. Bản chất của không gian địa chỉ khi nhiều người cùng chạy một tệp thực thi. Mỗi lần người dùng chạy một tệp .exe, hệ điều hành tạo ra một tiến trình mới với PCB, bộ đếm chương trình và bảng trang riêng. Vì vậy stack, heap và vùng data của từng tiến trình là độc lập. Riêng vùng text có thể được chia sẻ ở chế độ chỉ đọc để tiết kiệm RAM. Điều xảy ra trong vùng data là mỗi tiến trình vẫn có bản sao logic riêng của các biến toàn cục và biến tĩnh; nếu muốn thật sự dùng chung dữ liệu ghi được, các tiến trình phải sử dụng shared memory hoặc memory-mapped file.

2. Ba tình huống chuyển trạng thái Running - Waiting - Ready trong thực tế. Ví dụ thứ nhất là trình soạn thảo đang chạy nhưng phải chờ người dùng gõ phím; khi bàn phím phát sinh ngắt, tiến trình quay lại Ready. Ví dụ thứ hai là trình duyệt gửi yêu cầu tải tệp, sau đó chờ mạng hoặc đĩa hoàn tất; khi dữ liệu về, tiến trình được đưa trở lại ready queue. Ví dụ thứ ba là ứng dụng gửi lệnh in hoặc truy xuất cơ sở dữ liệu; trong lúc chờ thiết bị I/O phản hồi, tiến trình ở Waiting và chỉ trở lại Ready khi sự kiện đã hoàn thành.

3. Hai hướng giảm chi phí chuyển ngữ cảnh. Thứ nhất, bộ định thời nên hạn chế việc chuyển CPU quá dày đặc bằng cách chọn quantum hợp lý, kết hợp ưu tiên và CPU affinity để giảm số lần flush cache hoặc TLB không cần thiết. Thứ hai, hệ điều hành và phần cứng có thể tối ưu thao tác lưu và khôi phục ngữ cảnh, chẳng hạn dùng cấu trúc PCB gọn hơn, gom ngắt I/O hoặc tận dụng cơ chế hỗ trợ thanh ghi để rút ngắn thời gian save/restore.

4. Cách bộ định thời dài hạn cân bằng tiến trình CPU-bound và I/O-bound. Nếu hệ thống có quá nhiều tiến trình CPU-bound, CPU luôn bận nhưng các tiến trình tương tác phản hồi chậm; ngược lại, nếu chủ yếu là I/O-bound thì CPU dễ nhàn rỗi vì nhiều tiến trình

thường xuyên chờ thiết bị. Vì vậy long-term scheduler phải chọn một tỷ lệ pha trộn hợp lý để khi nhóm I/O-bound bị block thì nhóm CPU-bound vẫn khai thác CPU, còn khi CPU-bound bị chia lát thời gian thì các tiến trình tương tác vẫn giữ được độ đáp ứng.

5. Khác biệt giữa tiến trình con chỉ dùng fork() và tiến trình con gọi exec() ngay sau đó. Sau fork(), tiến trình con gần như là bản sao của tiến trình cha về không gian địa chỉ logic, mã lệnh và các mô tả tài nguyên; trên UNIX hiện đại việc nhân bản này thường được tối ưu bằng copy-on-write. Nếu tiến trình con gọi exec() ngay sau đó, toàn bộ vùng text, data, heap và stack cũ sẽ bị thay bằng chương trình mới, nhưng PID của tiến trình con vẫn được giữ nguyên. Điều này cho thấy fork() dùng để tạo tiến trình, còn exec() dùng để thay ảnh tiến trình bằng một nhiệm vụ khác.

6. Giá trị của biến pid và hiện tượng khi quên wait(NULL). Trong đoạn mã minh họa, pid < 0 biểu thị lỗi tạo tiến trình, pid == 0 là nhánh của tiến trình con, còn pid > 0 là nhánh của tiến trình cha và giá trị cụ thể chính là PID của con. Nếu tiến trình cha quên gọi wait(NULL), thứ tự hiển thị trên terminal sẽ không còn được đảm bảo: thông điệp của cha có thể xuất hiện trước hoặc sau kết quả của lệnh ls, và tiến trình con sau khi kết thúc có thể tồn tại tạm thời ở trạng thái zombie cho đến khi được cha hoặc hệ thống thu hồi.

7. Rủi ro của Zombie, cách hệ điều hành xử lý Orphan và ý nghĩa quản trị tài nguyên. Zombie không còn chạy nhưng vẫn giữ lại mục trong bảng tiến trình để lưu exit status; nếu xuất hiện hàng loạt, chúng có thể làm cạn PID hoặc làm đầy process table, từ đó cản trở việc tạo tiến trình mới. Trường hợp orphan thì ngược lại: khi tiến trình cha kết thúc sớm, hệ điều hành sẽ re-parent tiến trình con cho init hoặc systemd (PID = 1), sau đó tiến trình đặc biệt này sẽ gọi wait() khi tiến trình con chấm dứt để giải phóng tài nguyên còn sót lại.

8. Vì sao Google Chrome chọn kiến trúc đa tiến trình. Chrome chấp nhận chi phí RAM lớn hơn để đổi lấy ba lợi ích quan trọng: cô lập lỗi, tăng an toàn và quản lý tài nguyên rõ ràng theo từng tab. Nếu một renderer bị treo, hệ điều hành chỉ cần kết thúc đúng tiến trình đó thay vì làm sập toàn bộ trình duyệt. Đồng thời, cơ chế sandbox giới hạn quyền của renderer khiến mã độc trong một tab khó leo thang đặc quyền sang hệ thống hoặc các tab khác. Đây là ví dụ điển hình của việc ưu tiên độ ổn định và bảo mật hơn chi phí bộ nhớ thuần túy.

9. Lựa chọn mô hình IPC cho dữ liệu lớn và hệ thống nhắn tin. Với bài toán hai tiến trình cùng thao tác trên khối dữ liệu lớn như dựng hoặc render video 4K trong cùng một máy, shared memory thường cho hiệu suất tốt hơn vì dữ liệu không phải sao chép nhiều lần qua kernel. Ngược lại, với hệ thống chat nội bộ, message passing phù hợp hơn do thông điệp nhỏ, rời rạc, dễ kiểm soát, dễ mở rộng qua mạng và ít rủi ro ghi đè lên dữ liệu dùng chung. Nói cách khác, shared memory tối ưu băng thông nội bộ, còn message passing tối ưu sự mô-đun và độ an toàn giao tiếp.

10. Vì sao lời giải vùng đệm vòng chỉ dùng tối đa BUFFER_SIZE - 1 phần tử. Khi chỉ dùng hai biến in và out, trạng thái in == out được quy ước là buffer rỗng. Nếu cho phép ghi đầy toàn bộ BUFFER_SIZE ô, sau lần tăng vòng tiếp theo ta lại có in == out, khiến hệ thống không phân biệt được buffer đang rỗng hay đang đầy. Vì vậy giải pháp cơ bản phải chừa ra một ô để loại bỏ sự nhập nhằng này. Muốn khai thác đủ BUFFER_SIZE phần tử, chương trình phải bổ sung biến đếm, cờ full hoặc cơ chế semaphore và mutex để theo dõi chính xác số phần tử hiện có.

11. Truyền thông trực tiếp và gián tiếp khi hệ thống mở rộng. Truyền thông trực tiếp thích hợp khi hai tiến trình đã biết rõ nhau và quan hệ giao tiếp ổn định, vì cú pháp đơn giản và chi phí trung gian thấp. Tuy nhiên mô hình này tạo ra sự phụ thuộc chặt giữa bên gửi và bên nhận. Trái lại, truyền thông gián tiếp qua mailbox hoặc port làm giảm mức ghép nối, cho phép nhiều tiến trình cùng gửi và nhận, hỗ trợ cân bằng tải và thay thế dịch vụ dễ hơn khi hệ thống lớn dần. Do đó trong các kiến trúc nhiều thành phần, hàng đợi công việc hoặc microservices, truyền thông gián tiếp thường linh hoạt hơn.

Nhìn chung, phần thảo luận cho thấy cùng một khái niệm của hệ điều hành có thể dẫn đến các quyết định thiết kế rất khác nhau tùy theo mục tiêu ưu tiên là hiệu năng, khả năng mở rộng, độ an toàn hay tính đơn giản trong lập trình.

KẾT LUẬN

Quản lý tiến trình và truyền thông liên tiến trình là hai nội dung cốt lõi, quyết định đến hiệu năng và độ tin cậy của một hệ điều hành. Thông qua việc phân tích khái niệm tiến trình, cấu trúc, vòng đời và các cơ chế định thời, có thể thấy rằng hệ điều hành không chỉ đơn thuần là một chương trình điều khiển phần cứng, mà còn là một hệ thống phức tạp chịu trách nhiệm điều phối hoạt động của nhiều tiến trình cùng lúc.

Bên cạnh đó, các cơ chế truyền thông liên tiến trình như bộ nhớ chia sẻ và truyền thông thông điệp cho phép các tiến trình cộng tác hiệu quả, từ đó nâng cao khả năng xử lý song song và tối ưu hóa tài nguyên hệ thống. Mỗi mô hình IPC đều có ưu điểm và hạn chế riêng, đòi hỏi hệ điều hành cũng như lập trình viên phải lựa chọn giải pháp phù hợp với từng bài toán cụ thể.

Trong các hệ thống chia sẻ thời gian hiện đại, nơi nhiều người dùng và ứng dụng cùng hoạt động đồng thời, việc quản lý tiến trình hiệu quả giúp giảm thời gian chờ, nâng cao mức độ phản hồi và đảm bảo tính công bằng trong việc phân phối tài nguyên. Do đó, nghiên cứu và hiểu rõ cơ chế quản lý tiến trình và IPC không chỉ có ý nghĩa về mặt lý thuyết, mà còn là nền tảng quan trọng cho việc phát triển các hệ thống phần mềm và hệ điều hành trong thực tế.

TÀI LIỆU THAM KHẢO

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson Education, 2015.

[3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Boston, MA, USA: Pearson, 2018.

[4] Linux man-pages Project, "fork(2) — Linux manual page," Available: https://man7.org/linux/man-pages/man2/fork.2.html.

[5] Chromium Blog, "Multi-process Architecture," Available: https://blog.chromium.org/2008/09/multi-process-architecture.html.

[6] Microsoft Learn, "About Processes and Threads," Available: https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads.